CS 591 Lab Notebook

Lab 2

Name: Jarren Calizo
OdinID: calizo
Date: April 20, 2025

Table of Contents

1. *Lab Overview* 2

2. *Lab 2.1*
    - Linux File System Fundamentals – TryHackMe Walkthrough Completion 3
    - Exploring Linux Operators and File Ownership 5
    - Using grep and Extended Regular Expressions 7
    - Matching Character Occurrences with Regex 8
    - Matching Wildcards and Character Sets with grep 10
    - Using find to Identify Recently Modified Files 12
    - Executing Commands with -exec and xargs 13
    - Tampering with find via Timestamp Manipulation 15
    - Polyglot Files and Content-Type Heuristics 18
    - Investigating Polyglots via file, xxd, and hexeditor 19
    - TryHackMe Linux File System Forensics 21

3. Lab 2.2
    - Account Creation, UID/GID Enumeration, and Privilege Check 22
    - Understanding umask and Default File Permissions 24
    - Modifying File and Directory Permissions with chmod 25
    - Group Switching and chgrp Ownership Reassignment 26
    - Exploring setuid Permissions and Sudo Behavior 27
    - Investigating Shadow File Permissions & Using sudo/chown 28
    - Processes and /proc Exploration 29
    - Process Trees and Orphaned Execution in Linux 31
    - PATH Hijacking and Cron Script Evasion 32
    - sshd – SSH Configuration Analysis 34
    - Linux Process Analysis 36

Lab Overview

Lab 2.3 focused on exploiting XML External Entity (XXE) injection vulnerabilities in modern web applications and understanding how improperly sanitized XML parsers can become serious security liabilities. Across five progressive steps, I conducted both manual and scripted exploitation techniques ranging from basic file reads to remote request forgeries and image-based payload exfiltration. This lab served as a deep dive into how XML parsers behave when external entities are improperly processed, and how adversaries can manipulate server-side behavior by injecting malicious document type definitions (DTDs).

In the first stage, I manually examined an XML POST payload with Developer Tools, then mirrored that behavior using a Python script to simulate stock-check requests. This reinforced the fundamentals of request crafting and helped me validate how the application handled XML parsing. I then escalated to a full-blown XXE payload targeting /etc/passwd, successfully retrieving its contents through entity expansion using a scripted POST request.

From there, I leveraged XXE for server-side request forgery (SSRF) by targeting the AWS metadata service at http://169.254.169.254, exfiltrating IAM credentials through an indirect request. This showcased how insecure XML parsing could be abused to pierce internal network boundaries.

Next, I explored XInclude-based injection, where payloads were passed as form values and embedded into server-side XML via field interpolation. I inserted <xi:include> tags referencing sensitive files like /etc/passwd and confirmed that error-handling behavior echoed the target file's contents—validating the presence of a vulnerable parser chain.

Steps 4 and 5 covered the most creative attack vector—XXE via SVG file uploads. I crafted an SVG image containing a payload that displayed either my Odin ID (Step 4) or the system's hostname (Step 5) using entity expansion. These SVGs were uploaded as avatars, processed server-side using Apache Batik, and converted to PNGs which retained the injected payload as rendered text. I opened the avatar URL in-browser to verify the content, extracted the hostname visually, and submitted it to solve the final challenge.

This lab not only sharpened my understanding of XML parsing risks but also emphasized real-world implications like insecure image processing pipelines and SSRF risks from internal network access. These techniques bridge both web and cloud attack surfaces—areas I'm particularly interested in mastering further throughout this course.

Lab 2.1 Linux File System

Linux File System Fundamentals – TryHackMe Walkthrough Completion

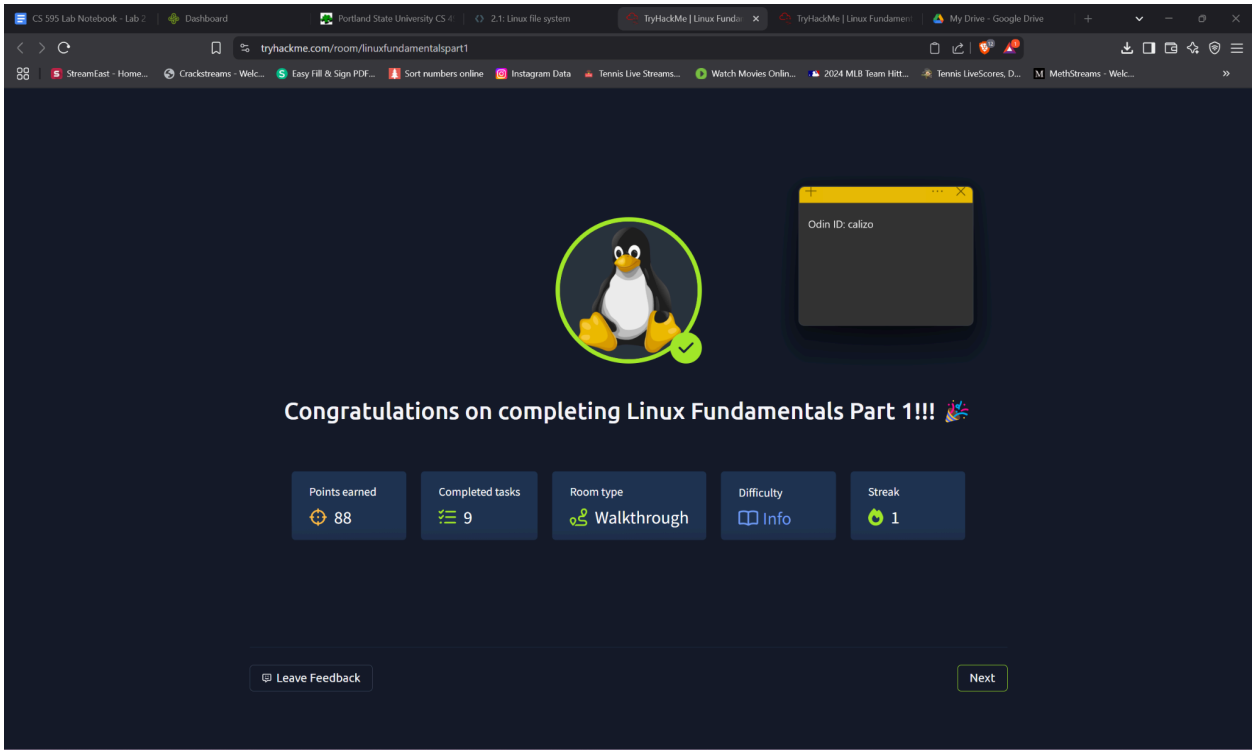
As part of Step 1 of Lab 2.1, I completed TryHackMe's Linux Fundamentals Part 1, an interactive walkthrough that introduces essential Linux skills foundational for both offensive and defensive security workflows. The room serves as a hands-on sandbox for developing confidence with the terminal and core filesystem navigation techniques. This marks my first official lab machine interaction within the course.

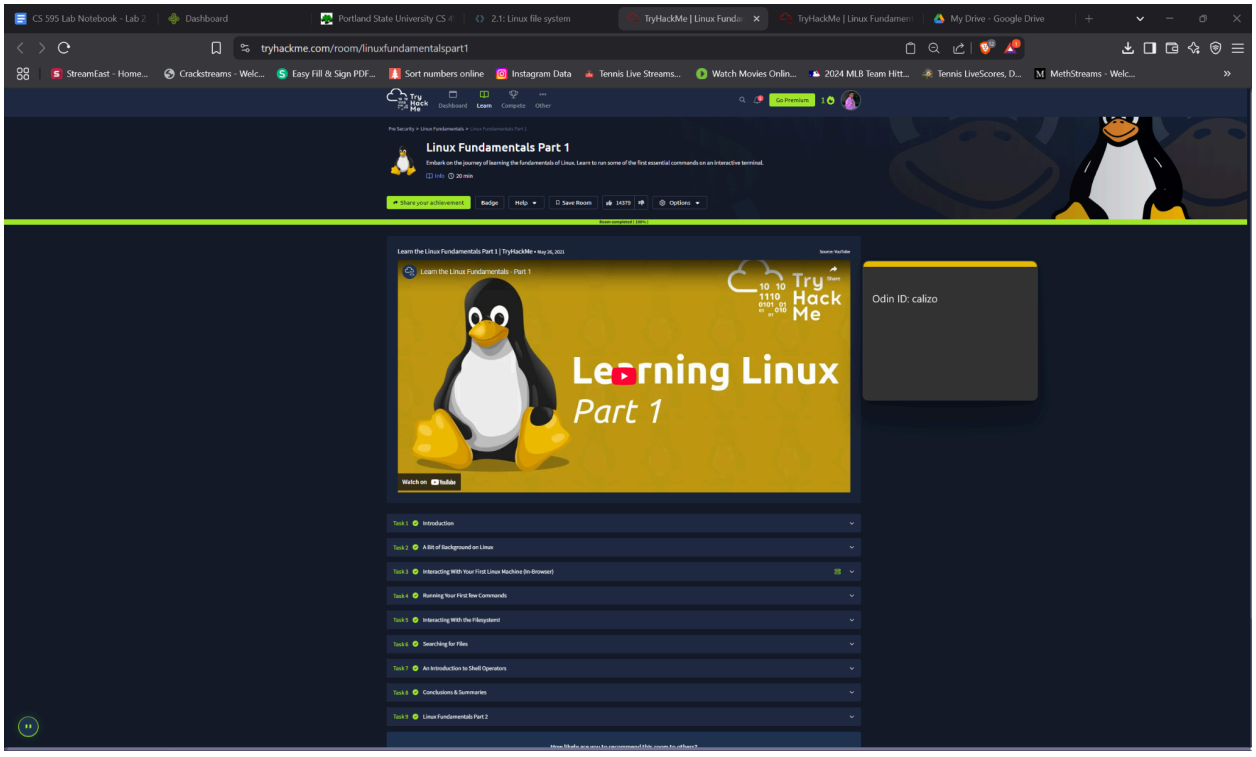
Key Concepts Practiced:

- Terminal Basics: I practiced echo, whoami, and pwd to output text, confirm user identity, and track the current working directory.
- Filesystem Navigation: Used ls, cd, and cat to explore and interact with directory contents. Practiced inspecting specific text files for important data (e.g., flags or configuration notes).
- Search Tools: Applied find to recursively locate files such as passwords.txt, and used grep to extract specific values (e.g., THM{} flags) from within log files. These are essential reconnaissance tools for any file-based CTF or forensic task.
- Output Redirection & Shell Operators:
    - > for overwriting file contents,
    - >> for appending data,
    - & to run commands in the background, and
    - && to chain commands conditionally.

Commands were practiced in the context of real-world workflows such as log analysis (access.log), redirection into configuration files, and building familiarity with recursive file searches in multi-level directories.

Attached are screenshots showing full room completion, along with my Odin ID ("calizo") as verification:





This lab serves as a foundation for more advanced Unix privilege escalation, automation, and system auditing topics to follow. I now feel more efficient moving through Linux environments using the terminal exclusively—an essential skill when

interfacing with headless servers, SSH shells, or containerized environments in later stages of this course

Exploring Linux Operators and File Ownership

This step focused on using common Linux shell tools to perform filtered inspection of filesystem contents — an essential practice for rapid triage, log analysis, and forensic auditing on Unix-based systems.

To begin, I practiced using the pipe (|) operator, which takes the standard output of one command and feeds it into the standard input of the next. This is useful for chaining commands in a logical flow without writing to intermediate files.
Specifically, I ran:
ls -l /var/log | grep auth

This command efficiently filtered the /var/log directory to locate all entries related to authentication logging, revealing auth.log and several archived versions (auth.log.1 to auth.log.4.gz). These files are commonly monitored for brute-force login attempts, sudo abuses, or other access control events.

Next, I explored the head utility, which returns the top portion of output or file content. Running:
ls -l /var/log | head -n 5

allowed me to preview the first five log entries — a helpful command for quickly scanning oversized log directories, especially when working over SSH where bandwidth or screen space is limited.

To demonstrate exclusion filtering, I ran:
ls -l /var/log | grep -v root

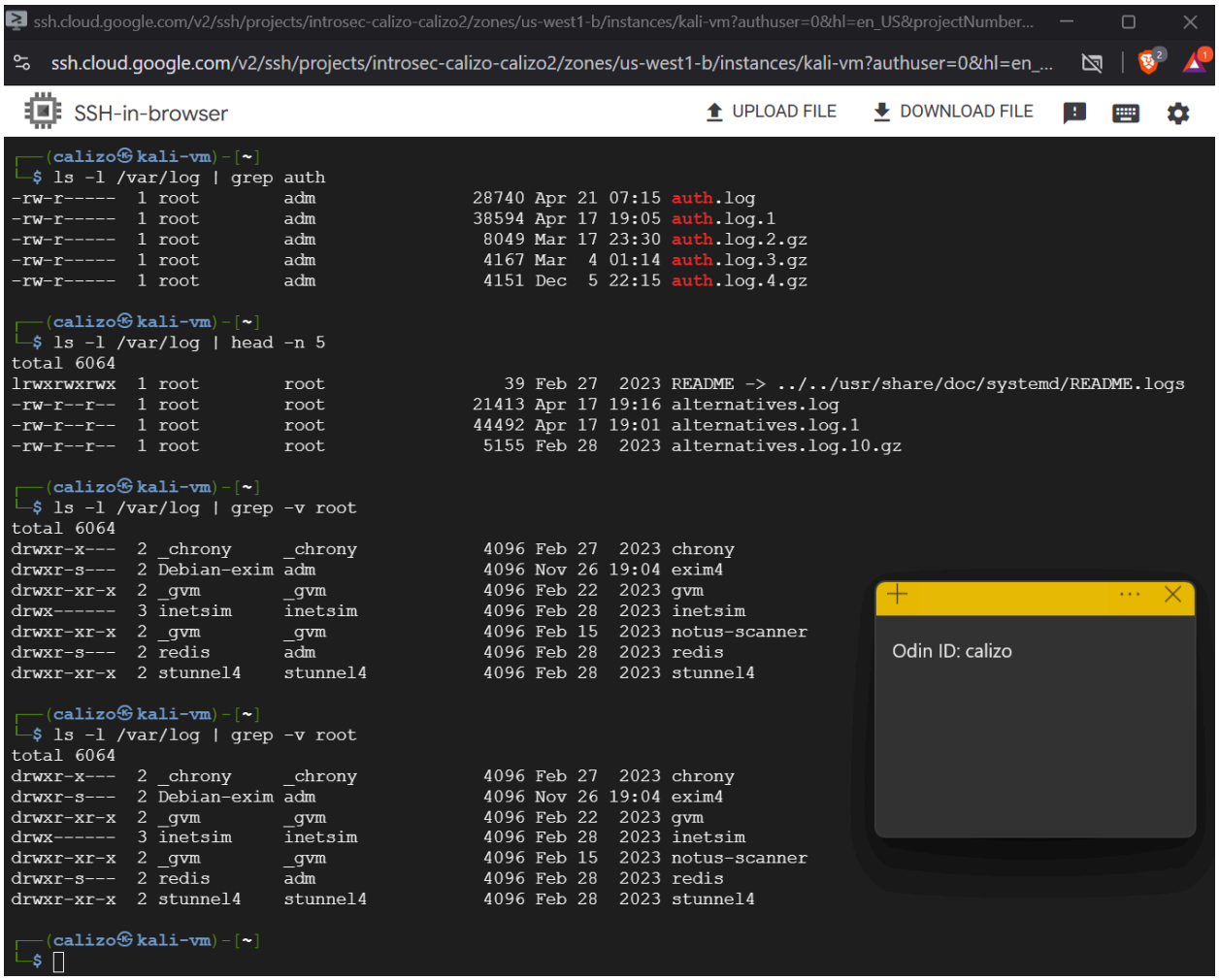
Using grep -v, I filtered out all entries owned by the root user to reveal logs created or managed by other system services. This revealed several directories owned by auxiliary users:

- chrony (time synchronization)
- Debian-exim (mail transport agent)
- gvm (Greenbone Vulnerability Management)
- inetstim (security scanning)
- notus-scanner (security scanning)
- redis (key-value database)

- stunnel4 (TLS/SSL tunneling)

These entries illustrate how compartmentalized logging is across services in a Linux environment. Recognizing log ownership is crucial in access control auditing — a non-root-owned log may expose sensitive data if improperly permissioned.

Commands and outputs from the Kali terminal are shown clearly in the attached screenshot:



This step reinforced the value of command-line fluency in log triage and service inspection. The ability to trace activity by filtering log files is foundational for both intrusion detection and vulnerability management workflows — a skill I expect to refine further as we build on privilege escalation and forensic recovery in later labs.

Using grep and Extended Regular Expressions

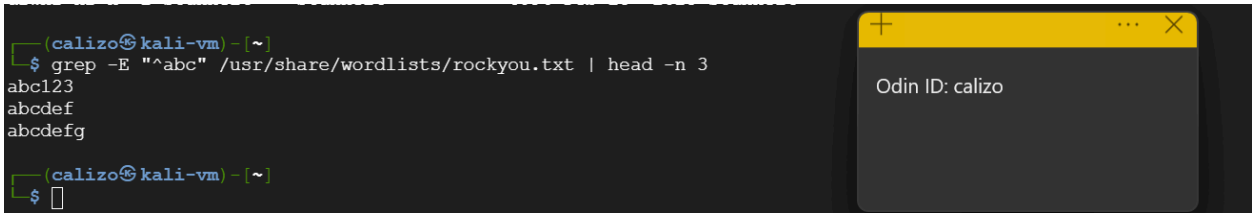
In this step, I explored the power of grep paired with extended regular expressions to filter plaintext passwords from the rockyou.txt wordlist—a widely used dataset sourced from a real-world breach. The exercise focused on boundary-matching regex operators ^ (start of string) and $ (end of string), both of which are foundational when crafting pattern-specific filters.

Task 1: Match passwords beginning with "abc"
To locate the first 3 passwords starting with the characters abc, I combined grep -E (for extended regex) with head to truncate the output:
grep -E "^abc" /usr/share/wordlists/rockyou.txt | head -n 3

This command:

- Searched the file for lines starting with abc
- Returned only the first 3 matches

The output confirmed the following matches:
abc123
abcdef
abcdefg

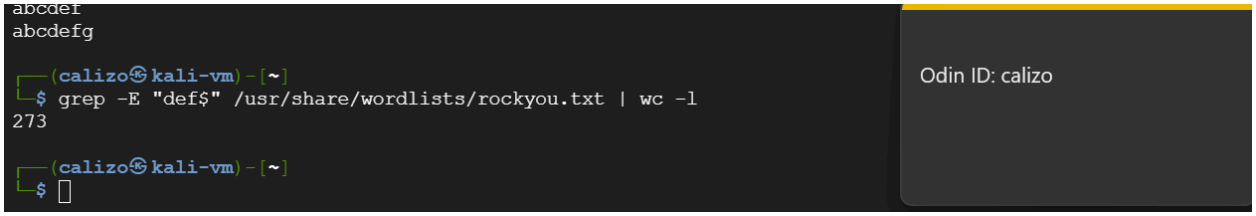


Task 2: Count passwords ending with "def"
To identify the number of passwords that ended specifically with def, I used the $ anchor to match string endings and piped the result to wc -l for a count:
grep -E "def$" /usr/share/wordlists/rockyou.txt | wc -l

- This expression isolated all exact line matches ending in def.

Output:
273

This indicates there are 273 unique passwords in the RockYou list that terminate with the substring "def".



This task demonstrated the efficiency of command-line tools like grep when paired with regex anchors in high-volume datasets. These string boundary operators are particularly useful in password policy enforcement, pattern-based password spray detection, and log parsing scenarios where filtering by start/end constraints can reduce search space significantly.

Matching Character Occurrences with Regex

This lab step explored regex quantifiers and their ability to precisely match patterns based on character frequency and positioning. Applied against the rockyou.txt password dump file, these commands simulate how real-world password cracking, credential validation, or anomaly detection engines might operate on large datasets.

Task 1: Match character frequencies using ?, +, *
1A. ^abcde(f)?$ → Match strings starting with abcde and ending with either nothing or a single f

grep -E "^abcde(f)?$" /usr/share/wordlists/rockyou.txt

- Matches: abcde, abcdef
- Doesn't match: abcdeff

1B. ^abcde(f)+$ → Match strings starting with abcde and ending with one or more fs

grep -E "^abcde(f)+$" /usr/share/wordlists/rockyou.txt

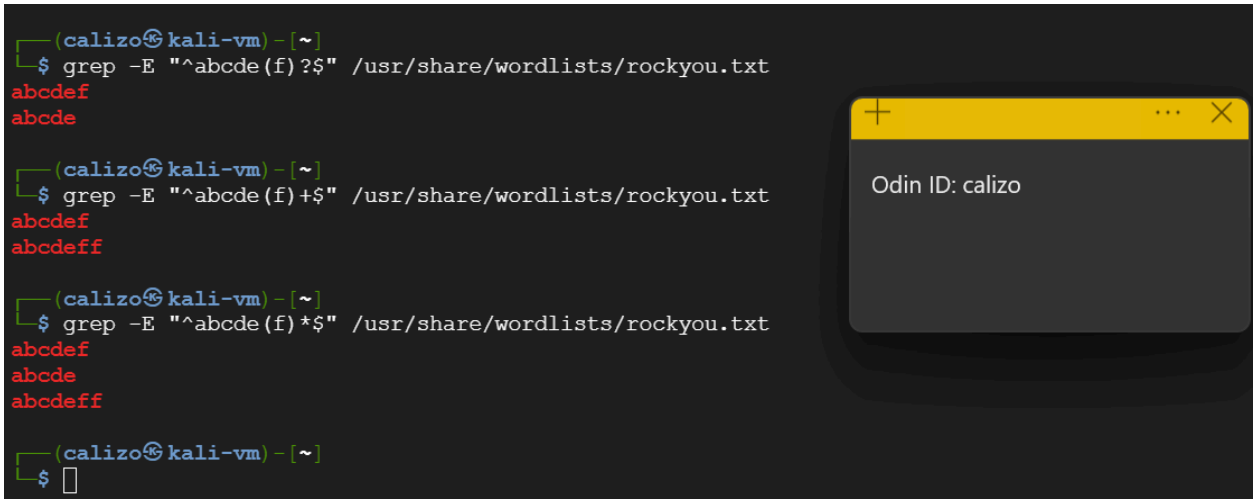
- Matches: abcdef, abcdeff
- Doesn't match: abcde (no trailing f)

1C. ^abcde(f)*$ → Match strings starting with abcde and ending with zero or more f's

grep -E "^abcde(f)*$" /usr/share/wordlists/rockyou.txt

- Matches: abcde, abcdef, abcdeff

Screenshot shows all three valid variants (1A, 1B, 1C) printed correctly:



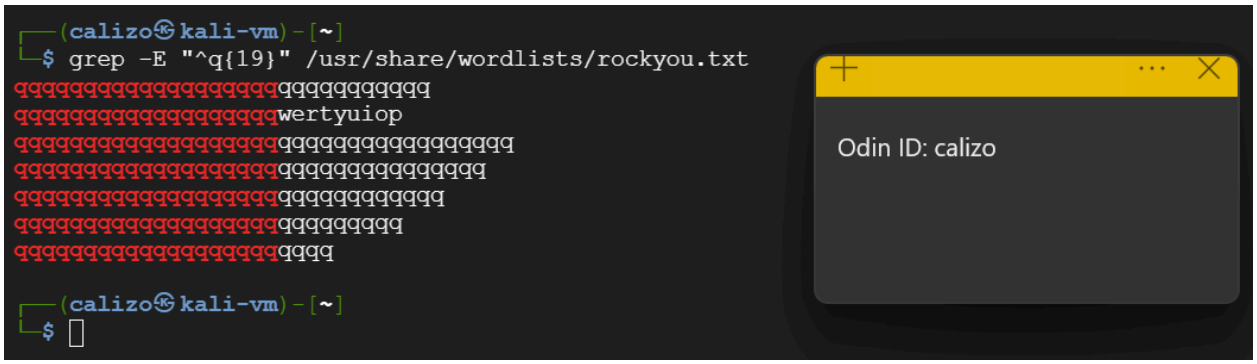
These patterns reinforced my understanding of how greedy quantifiers function in shell-based regex engines and how they relate to data validation, brute-force rulesets, and signature-based detection in real-world tooling.

🔬 Task 2: Match strings beginning with exactly 19 qs using {n}

grep -E "^q{19}" /usr/share/wordlists/rockyou.txt

- This command searches for any string beginning with exactly 19 q characters. Several matches were returned, including longer strings with more than 19 qs and embedded characters after the base match.
- Output includes several long q-heavy strings like:
  - qqqqqqqqqqqqqqqqqqqqqqqqqqqqqq
  - qqqqqqqqqqqqqqqqqqqqwertyuiop
  - ...

Screenshot taken, displaying all output and Odin ID for validation:



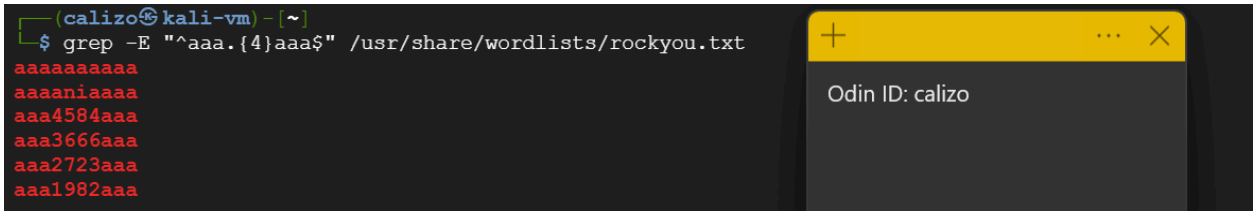
These regex-driven tasks helped reinforce not only command syntax but also sharpened pattern recognition against large wordlists. These filtering techniques scale

into offensive password cracking rules and defensive heuristics in tools like Fail2Ban, intrusion detection systems (IDS), and password policy auditors.

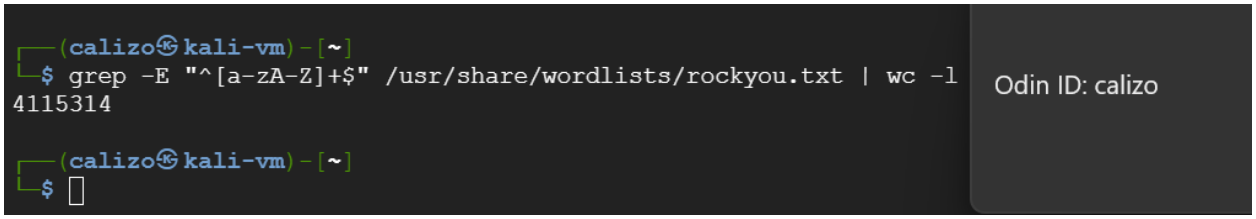
Matching Wildcards and Character Sets with grep

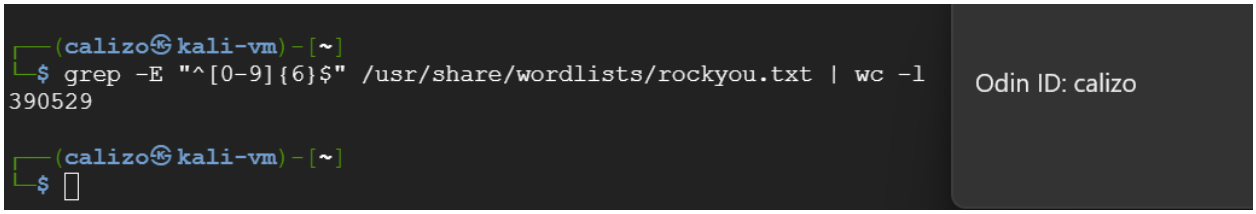
This step focused on exploring the flexibility and power of wildcard and character class matching in Linux regular expressions. I used grep -E to evaluate a series of increasingly nuanced search patterns across the rockyou.txt wordlist, simulating real-world use cases like password policy enforcement, brute-force rule design, and character entropy analysis.

The first task involved filtering for exactly 10-character passwords that begin and end with three 'a' characters, using the pattern ^aaa.{4}aaa$. This pattern leverages positional anchors, a fixed-length wildcard segment, and exact character placement. Matches such as aaaaaaaaaa, aaa4584aaa, and aaaani1aaa validated the pattern’s ability to precisely isolate expected structures.

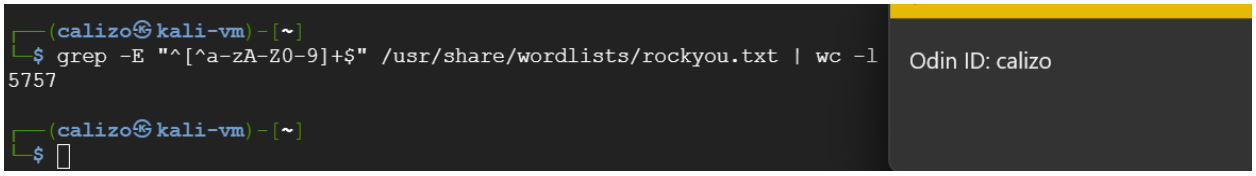


Next, I used bracketed character classes to measure the prevalence of alphabet-only passwords. The command grep -E "^[a-zA-Z]+$" filtered entries made entirely of upper- or lowercase letters, yielding a count of 4,115,314 unique alphabetic passwords—an important observation showing the high frequency of human-readable or dictionary-based entries in breached datasets. Following that, I narrowed scope to count numeric passwords that consist of exactly six digits using grep -E "^[0-9]{6}$". This revealed another significant volume of matches: 390,529. These results underscore how common minimal numeric passwords still are in real-world password leaks.

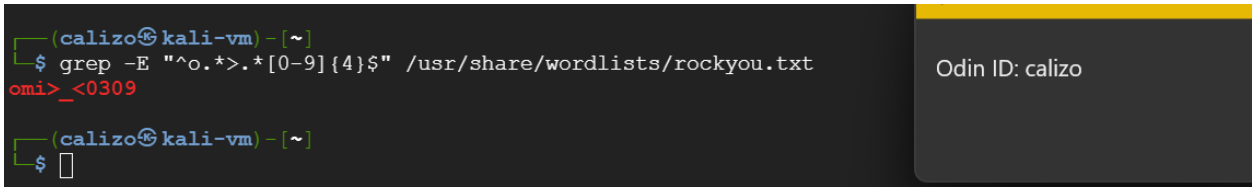




For a deeper dive into symbol-heavy entries, I filtered for passwords composed solely of non-alphanumeric characters via grep -E "^[^a-zA-Z0-9]+$", leveraging the negated character class ([^...]). This yielded only 5,757 results, a relatively rare pattern, reflecting the infrequency with which users choose exclusively symbolic passwords—either due to memorability concerns or system constraints.



The final challenge was more targeted. I needed to identify a specific password that started with the letter o, contained a > character somewhere in the middle, and ended with exactly four digits. The pattern ^o.*>.*[0-9]{4}$ stitched together each of those constraints into a single query. The output revealed a match: omi>_<0309, which perfectly satisfied the pattern. This reinforced how expressive and surgical regex becomes when combining anchors, greedy wildcards, and quantifiers in layered logic.



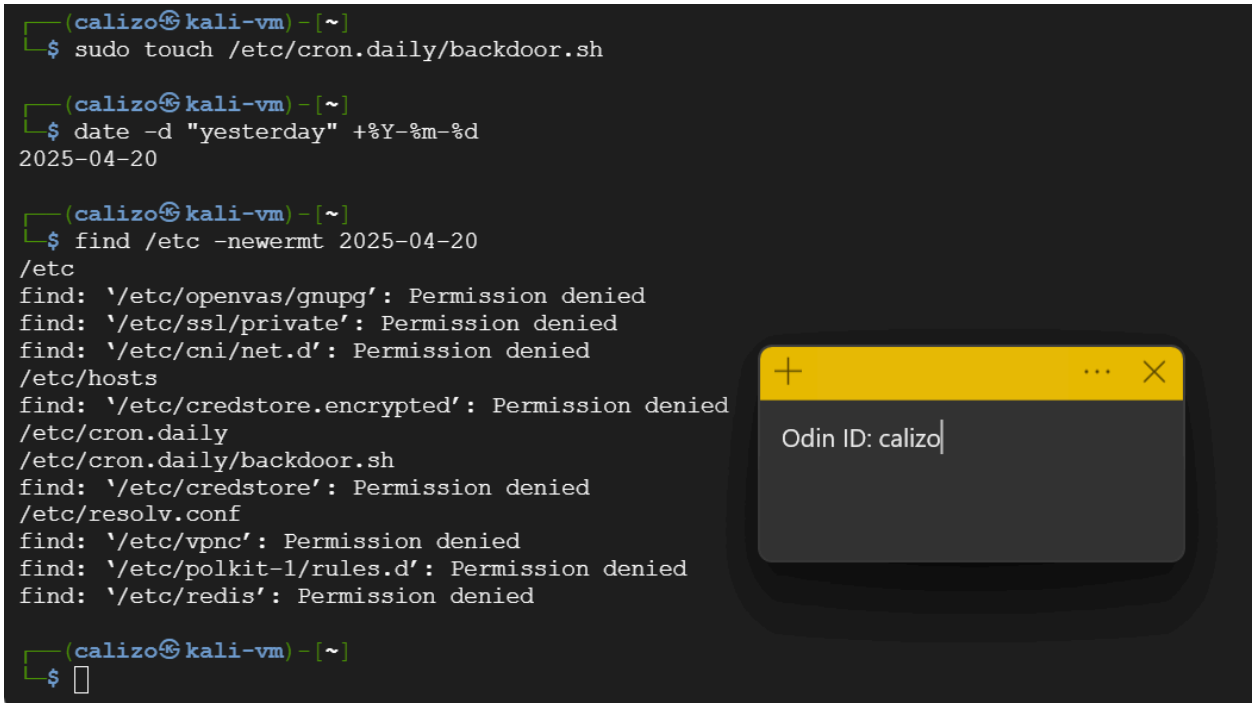
Overall, this lab deepened my understanding of how wildcard operators like ., repetition quantifiers like {n}, and bracketed character sets enable powerful and targeted string filtering—whether for data validation, password strength filtering, or forensic regex carving across massive log files. These are tools I can now apply with precision across shell pipelines, cybersecurity scripts, or intrusion detection filters.

Using find to Identify Recently Modified Files

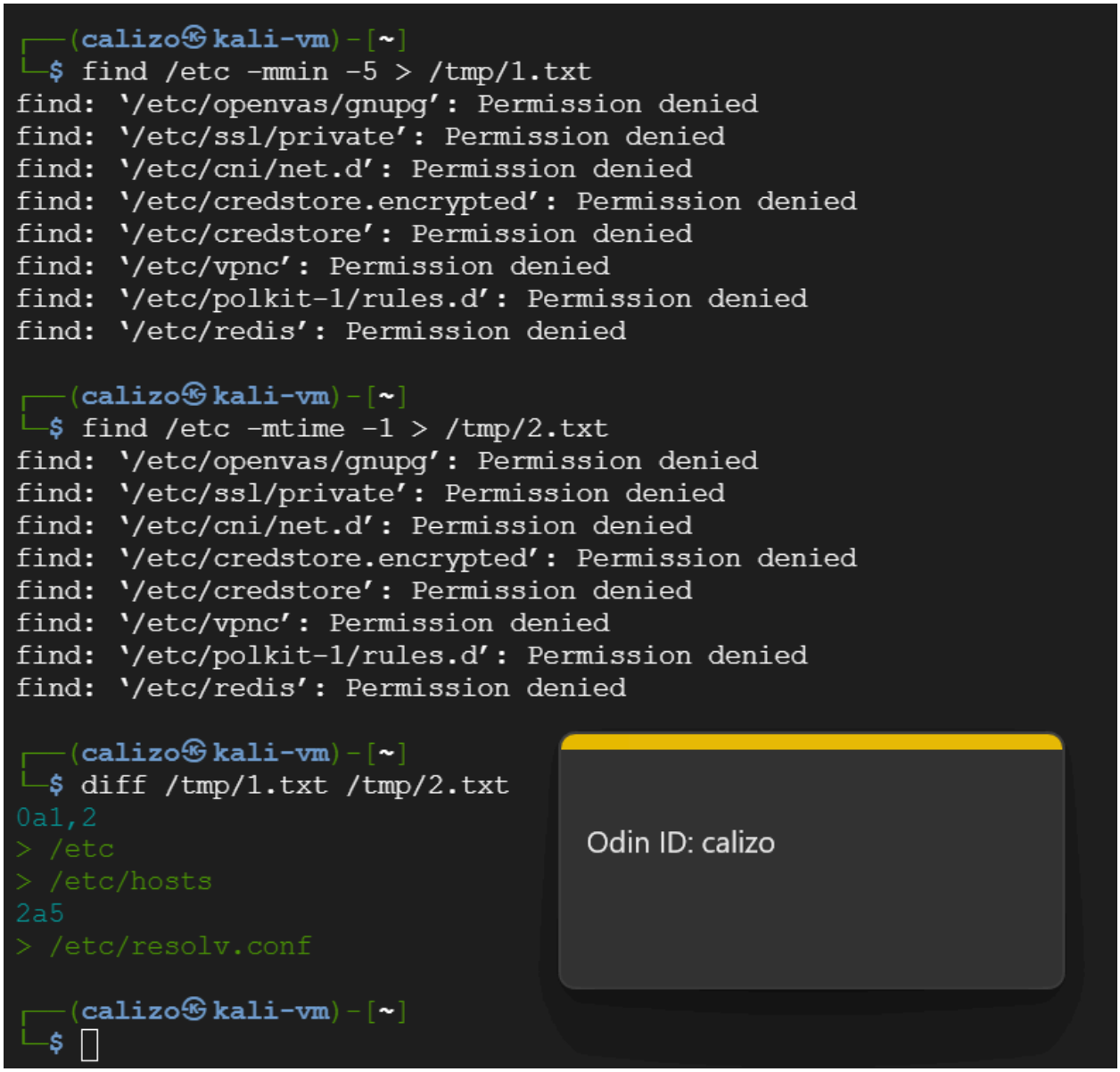
This step explored how the find command can be used in a forensic or system auditing context to locate recently modified files—critical for detecting unauthorized

configuration changes or persistence mechanisms introduced by an attacker. To simulate a scenario involving malicious cron job insertion, I manually created an empty file called backdoor.sh inside /etc/cron.daily/ using sudo touch. Since this directory houses scripts that are executed automatically once per day by cron, it’s an ideal place for attackers to plant persistent payloads.

To validate whether we could detect this tampering, I used the date -d "yesterday" +%Y-%m-%d command to generate the previous day’s date (2025-04-20), then used find /etc -newermt 2025-04-20 to search for files in /etc that had been modified more recently. As expected, the output included /etc/cron.daily/backdoor.sh, confirming that find with the -newermt flag can reliably flag filesystem changes. I also saw some "Permission denied" errors for protected subdirectories like /etc/ssl/private and /etc/redis, which is expected without elevated privileges.



For the second half of the task, I tested relative time-based filters using -mmin and -mtime. I first ran find /etc -mmin -5 > /tmp/1.txt to store all files modified in the last 5 minutes, then used find /etc -mtime -1 > /tmp/2.txt to capture all files modified within the last 24 hours. To identify files modified between those two ranges (i.e., more than 5 minutes ago but within 1 day), I used diff /tmp/1.txt /tmp/2.txt. The result showed that files like /etc/hosts and /etc/resolv.conf appeared in /tmp/2.txt but not in /tmp/1.txt, confirming their modification occurred sometime between 5 minutes and 1 day ago.



This exercise reinforced how effective find can be when combined with time-based flags for incident response, especially in scenarios where an attacker may have tampered with configuration files or left behind scheduled execution scripts. Understanding how to slice timestamps in this way allows a responder to narrow down the scope of affected files with surgical precision.

Executing Commands with -exec and xargs

In this lab step, I explored two of the most powerful post-processing techniques available with the find command: -exec and xargs. Both allow you to take results

returned by find and directly feed them into other commands, making it easy to batch-process files based on search criteria like modification time.

To simulate detection of modified files, I first used find /etc -type f -mtime -1 -exec sha256sum {} \; to hash every file in /etc that had been altered within the past 24 hours. The -type f filter ensured only regular files were considered, while -mtime -1 scoped the search to the last 1-day window. The -exec syntax executed sha256sum on each individual result using {} as a placeholder, escaping the trailing semicolon as \; to avoid premature shell interpretation. This command produced hash digests for recent changes, including /etc/hosts, /etc/resolv.conf, and the planted /etc/cron.daily/backdoor.sh file, verifying its presence in the results.

To showcase the efficiency of xargs for batch execution, I re-ran the file search using: find /etc -type f -mtime -1 | xargs ls -l

This fed all matching files into a single invocation of ls -l, listing permissions, timestamps, and ownership attributes in one clean view. Though some "Permission denied" messages appeared due to restricted subdirectories, the command correctly displayed details for the backdoor script and both system config files.

Finally, I refined the output by suppressing permission errors using:
find /etc -type f -mtime -1 2>/dev/null | xargs ls -l

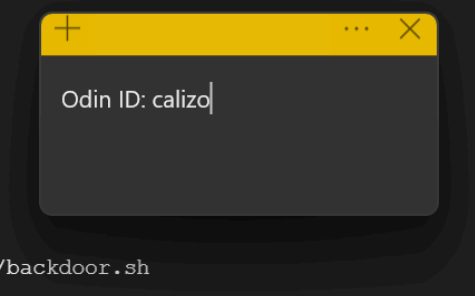
This adjustment eliminated noise, making the output concise and clean—particularly important when automating scans or redirecting results to log files or triage scripts.

```
┌──(calizo㉿kali-vm)-[~]
└─$ find /etc -type f -mtime -1 -exec sha256sum {} \;
find: '/etc/openvas/gnupg': Permission denied
find: '/etc/ssl/private': Permission denied
find: '/etc/cni/net.d': Permission denied
2caae18ccd767ea3efd12ab8e6bfb636820ab83eeaabd667c57b6c2e61d1929a  /etc/hosts
find: '/etc/credstore.encrypted': Permission denied
e3b0c44298fc1c149afbf4c8996fb92427ae41e4649b934ca495991b7852b855  /etc/cron.daily/backdoor.sh
find: '/etc/credstore': Permission denied
5e3b52c35d2c5cbdbc8c6970189a5fce680d62c44e3505c5ec37d17e2850c585  /etc/resolv.conf
find: '/etc/vpnc': Permission denied
find: '/etc/polkit-1/rules.d': Permission denied
find: '/etc/redis': Permission denied

┌──(calizo㉿kali-vm)-[~]
└─$ find /etc -type f -mtime -1 | xargs ls -l
find: '/etc/openvas/gnupg': Permission denied
find: '/etc/ssl/private': Permission denied
find: '/etc/cni/net.d': Permission denied
find: '/etc/credstore.encrypted': Permission denied
find: '/etc/credstore': Permission denied
find: '/etc/vpnc': Permission denied
find: '/etc/polkit-1/rules.d': Permission denied
find: '/etc/redis': Permission denied
-rw-r--r-- 1 root root   0 Apr 21 08:02 /etc/cron.daily/backdoor.sh
-rw-r--r-- 1 root root 250 Apr 21 07:07 /etc/hosts
-rw-r--r-- 1 root root 130 Apr 21 07:07 /etc/resolv.conf

┌──(calizo㉿kali-vm)-[~]
└─$ find /etc -type f -mtime -1 2>/dev/null | xargs ls -l
-rw-r--r-- 1 root root   0 Apr 21 08:02 /etc/cron.daily/backdoor.sh
-rw-r--r-- 1 root root 250 Apr 21 07:07 /etc/hosts
-rw-r--r-- 1 root root 130 Apr 21 07:07 /etc/resolv.conf

┌──(calizo㉿kali-vm)-[~]
└─$
```



Overall, this step emphasized the flexibility and composability of Unix tools. Whether performing digital forensics, auditing system integrity, or building reactive tooling, the ability to chain find, -exec, and xargs together provides a scalable way to process targeted file sets without writing temporary scripts or manual loops.

Tampering with find via Timestamp Manipulation

This step simulated a real-world anti-forensics tactic often used by attackers to avoid detection via file timestamp obfuscation. After reviewing various rootkit-based tampering strategies (kernel-mode, user-mode, and binary-level), the exercise focused on the most accessible of these techniques—timestamp manipulation—as a method of hiding malicious activity from standard find-based incident response workflows.

To begin, I recreated a mock persistence mechanism by deleting and recreating the file /etc/cron.daily/backdoor.sh. Since this directory is automatically executed by

cron daily, it represents a realistic target for stealthy payload execution. After re-creating the file, I ran find /etc -mmin -30 -exec ls -ld {} \; to confirm the file showed up in a typical recent modification search. As expected, it was returned in the results, along with /etc/cron.daily itself.

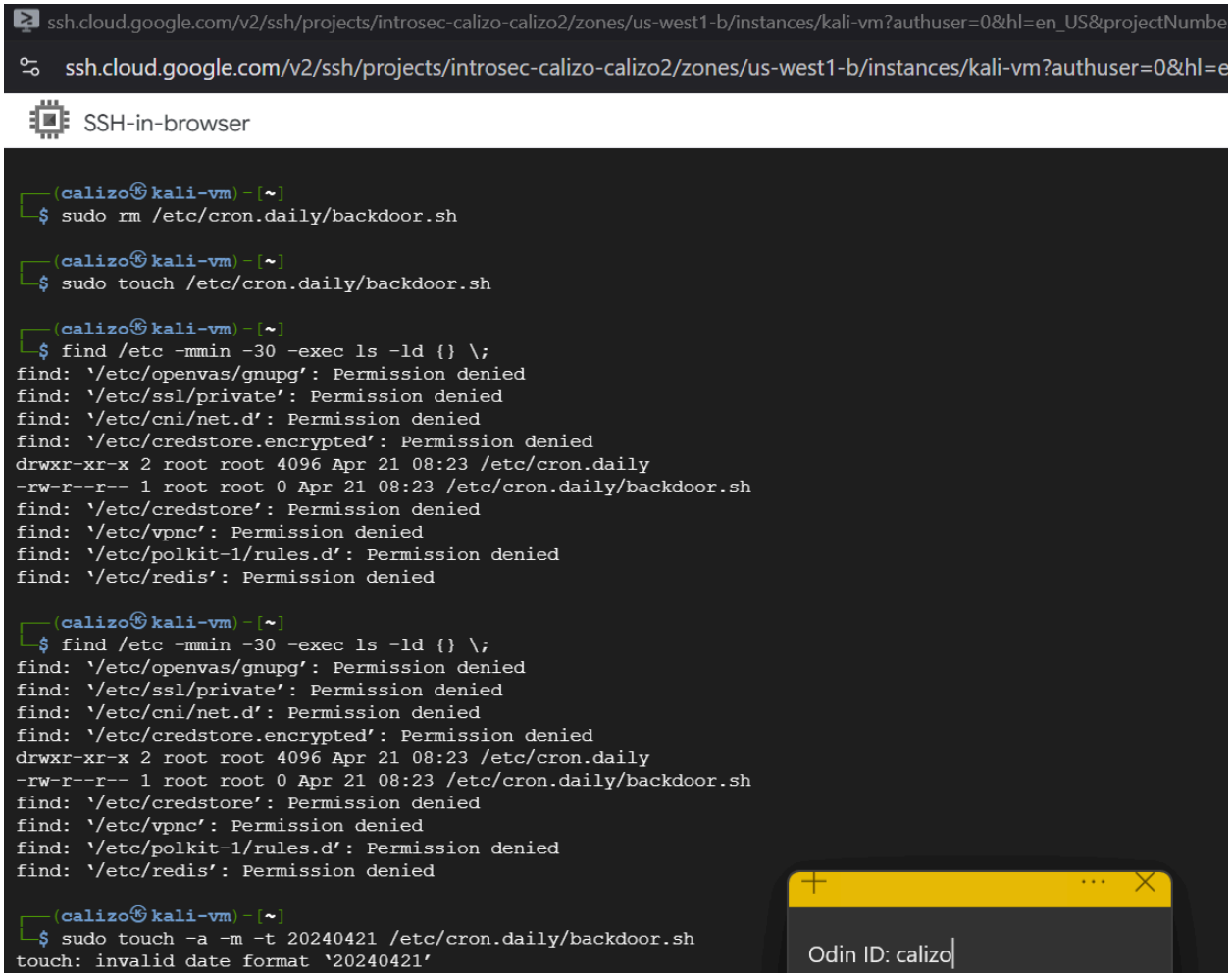
To explore tampering, I examined the man page for touch using man touch and found the following relevant flags:

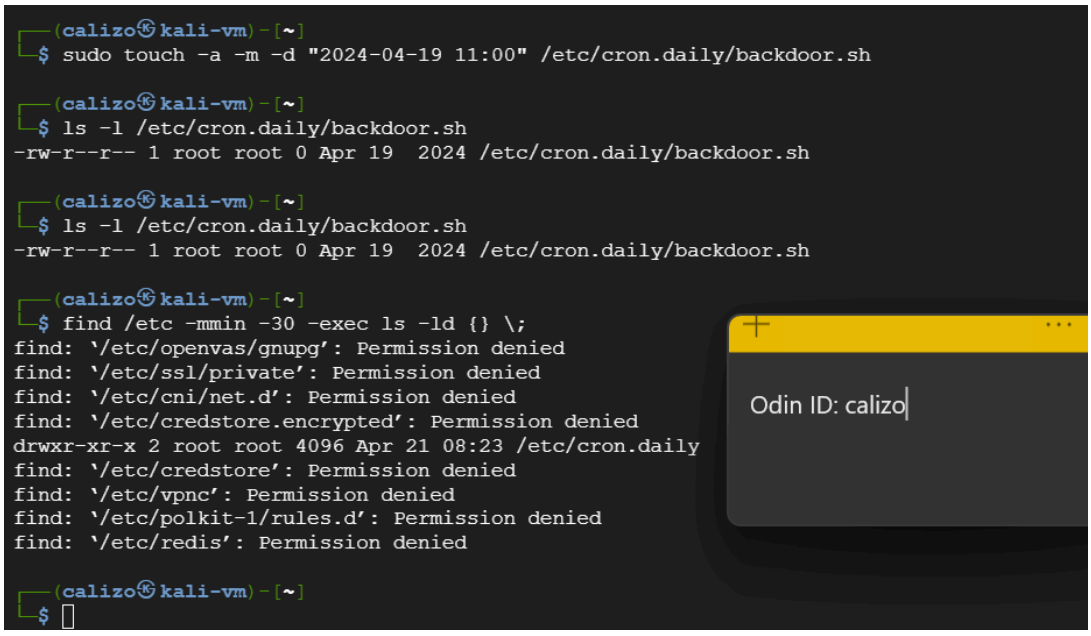
- **-a** is used to change the access time
- **-m** is used to change the modification time
- **-t** allows setting a custom timestamp in the format [[CC]YY]MMDDhhmm[.ss]

For example, -t 202404210000 represents April 21, 2024 at 00:00. Alternatively, I verified that the -d flag allows a more human-readable input such as -d "2024-04-21 00:00".

Using this knowledge, I changed the file’s timestamp with:
sudo touch -a -m -t 202404190000 /etc/cron.daily/backdoor.sh

After verifying the change via ls -l /etc/cron.daily/backdoor.sh, the timestamp was successfully set to April 19, 2024—a date outside of the current 30-minute search window. I re-ran the find /etc -mmin -30 -exec ls -ld {} \; command and confirmed that the backdoor.sh file was now omitted from the output.


ssh.cloud.google.com/v2/ssh/projects/introsec-calizo-calizo2/zones/us-west1-b/instances/kali-vm?authuser=0&hl=en_US&projectNumbe
ssh.cloud.google.com/v2/ssh/projects/introsec-calizo-calizo2/zones/us-west1-b/instances/kali-vm?authuser=0&hl=e
SSH-in-browser
(calizo㉿kali-vm)-[~]
$ sudo rm /etc/cron.daily/backdoor.sh
(calizo㉿kali-vm)-[~]
$ sudo touch /etc/cron.daily/backdoor.sh
(calizo㉿kali-vm)-[~]
$ find /etc -mmin -30 -exec ls -ld {} \;
find: '/etc/openvas/gnupg': Permission denied
find: '/etc/ssl/private': Permission denied
find: '/etc/cni/net.d': Permission denied
find: '/etc/credstore.encrypted': Permission denied
drwxr-xr-x 2 root root 4096 Apr 21 08:23 /etc/cron.daily
-rw-r--r-- 1 root root 0 Apr 21 08:23 /etc/cron.daily/backdoor.sh
find: '/etc/credstore': Permission denied
find: '/etc/vpnc': Permission denied
find: '/etc/polkit-1/rules.d': Permission denied
find: '/etc/redis': Permission denied
(calizo㉿kali-vm)-[~]
$ find /etc -mmin -30 -exec ls -ld {} \;
find: '/etc/openvas/gnupg': Permission denied
find: '/etc/ssl/private': Permission denied
find: '/etc/cni/net.d': Permission denied
find: '/etc/credstore.encrypted': Permission denied
drwxr-xr-x 2 root root 4096 Apr 21 08:23 /etc/cron.daily
-rw-r--r-- 1 root root 0 Apr 21 08:23 /etc/cron.daily/backdoor.sh
find: '/etc/credstore': Permission denied
find: '/etc/vpnc': Permission denied
find: '/etc/polkit-1/rules.d': Permission denied
find: '/etc/redis': Permission denied
(calizo㉿kali-vm)-[~]
$ sudo touch -a -m -t 20240421 /etc/cron.daily/backdoor.sh
touch: invalid date format '20240421'
Odin ID: calizo


(calizo㉿kali-vm)-[~]
$ sudo touch -a -m -d "2024-04-19 11:00" /etc/cron.daily/backdoor.sh
(calizo㉿kali-vm)-[~]
$ ls -l /etc/cron.daily/backdoor.sh
-rw-r--r-- 1 root root 0 Apr 19 2024 /etc/cron.daily/backdoor.sh
(calizo㉿kali-vm)-[~]
$ ls -l /etc/cron.daily/backdoor.sh
-rw-r--r-- 1 root root 0 Apr 19 2024 /etc/cron.daily/backdoor.sh
(calizo㉿kali-vm)-[~]
$ find /etc -mmin -30 -exec ls -ld {} \;
find: '/etc/openvas/gnupg': Permission denied
find: '/etc/ssl/private': Permission denied
find: '/etc/cni/net.d': Permission denied
find: '/etc/credstore.encrypted': Permission denied
drwxr-xr-x 2 root root 4096 Apr 21 08:23 /etc/cron.daily
find: '/etc/credstore': Permission denied
find: '/etc/vpnc': Permission denied
find: '/etc/polkit-1/rules.d': Permission denied
find: '/etc/redis': Permission denied
(calizo㉿kali-vm)-[~]
$
Odin ID: calizo

This demonstrated how a simple timestamp alteration can effectively hide files from basic time-based incident response commands. While more advanced defenses would rely on integrity verification (e.g., aide, tripwire, or kernel-level monitoring), this step reinforced how important it is to go beyond surface-level file inspection in forensics and blue-team ops.

Polyglot Files and Content-Type Heuristics

This lab focused on polyglot files—single files that can be interpreted as two different types depending on the parser or context. This technique can be weaponized in attacks such as masquerading executables as images or PDFs to bypass file upload filters or antivirus scans. The polyglot example in this lab was linked from the course server in two forms: once as a .gif (image) and once as a .pdf (document). Both point to the same binary data via a hard link but carry different file extensions.

To identify the creator of this polyglot, I opened either version of the file in a web browser and observed the metadata within the PDF structure. The author of this particular GIF/PDF polyglot is Hugo Laloge.

I then opened my Kali VM and used the curl -I command to fetch HTTP headers for both URLs:
curl -I https://thefengs.com/wuchang/courses/cs491/files/GIF+PDF.gif
curl -I https://thefengs.com/wuchang/courses/cs491/files/GIF+PDF.pdf

Both returned identical values for:

- **Content-Length: 973165**
- **Last-Modified: Wed, 25 May 2022 18:19:27 GMT**
- **ETag: "628e732f-ed96d"**

However, the one difference was in the Content-Type field:

- The .gif version returned: **Content-Type: image/gif**
- The .pdf version returned: **Content-Type: application/pdf**

This difference makes it clear that the web server uses a **file extension heuristic**—not a file content signature—to determine the MIME type for Content-Type. Despite both files being byte-for-byte identical, the only change in response was driven by their file name suffix.

This experiment demonstrated how trivial it can be to manipulate file-type detection systems, especially those relying on superficial checks like extensions or MIME headers. It reinforces the need for deeper file validation at the binary level when designing secure file upload pipelines or defending against evasive payloads.

Investigating Polyglots via file, xxd, and hexeditor

This step explored the behavior of the file utility in analyzing file formats based on internal signatures, not just superficial extensions. To begin, I downloaded a known polyglot from the course materials—GIF+PDF.gif—which is a dual-formatted file crafted to behave validly as both a GIF image and a PDF document depending on context. I saved the file to my Kali VM using:
curl -o polyglot.gif https://thefengs.com/wuchang/courses/cs491/files/GIF+PDF.gif

Running file polyglot.gif returned:
polyglot.gif: GIF image data, version 89a, 500 x 375

I then examined the file's internal structure using:
xxd polyglot.gif | head -2

The dump confirmed a valid GIF signature at the beginning (47 49 46 38 39 61 → GIF89a). As expected, file correctly reported the file as a GIF image.

Next, I renamed the file to polyglot.pdf and re-ran the file command:
mv polyglot.gif polyglot.pdf
file polyglot.pdf

Despite the new extension, the file still reported as a GIF image. This proves that file relies on content-based heuristics, specifically magic bytes embedded in the file header, rather than just using the file extension.

To alter its identity, I launched hexeditor:
hexeditor polyglot.pdf

and manually modified the first five bytes of the file to 25 50 44 46 2D—the ASCII hex values for %PDF-, which is the expected signature of a valid PDF document. After saving with Ctrl+X, I reran:
file polyglot.pdf

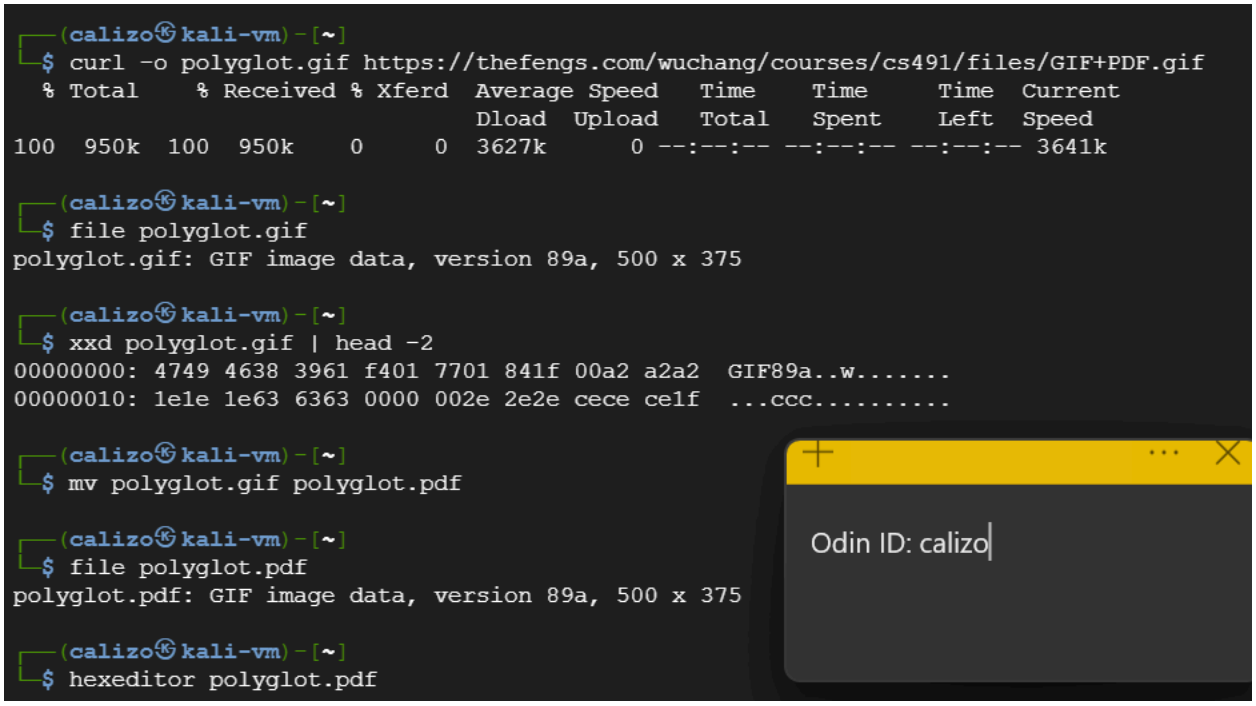
This time, the output changed to:

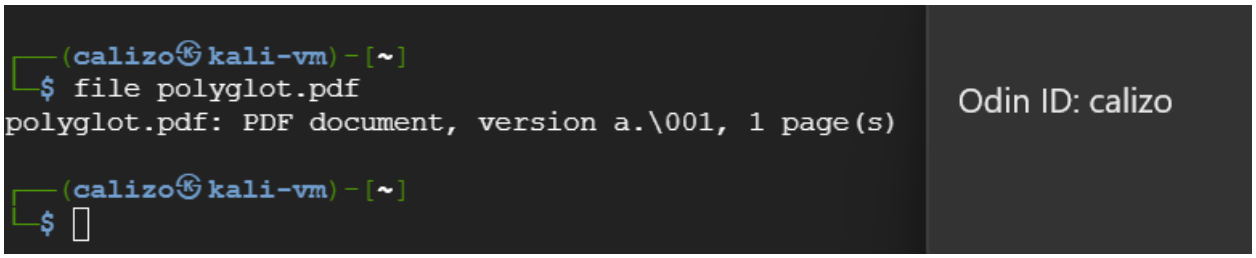
polyglot.pdf: PDF document, version a.\001, 1 page(s)

confirming the successful overwrite of the file's magic bytes and demonstrating how easily a file can spoof its identity to bypass content filters or deceive handlers.

In sum,
Does the file command use extension or content-based heuristics?
→ It uses **content-based heuristics**, relying on magic bytes at the file header rather than the filename extension.

Was the filetype successfully changed using hexeditor?
→ Yes. After changing the header bytes to %PDF-, the file was reclassified from a GIF to a valid PDF document by the file utility.





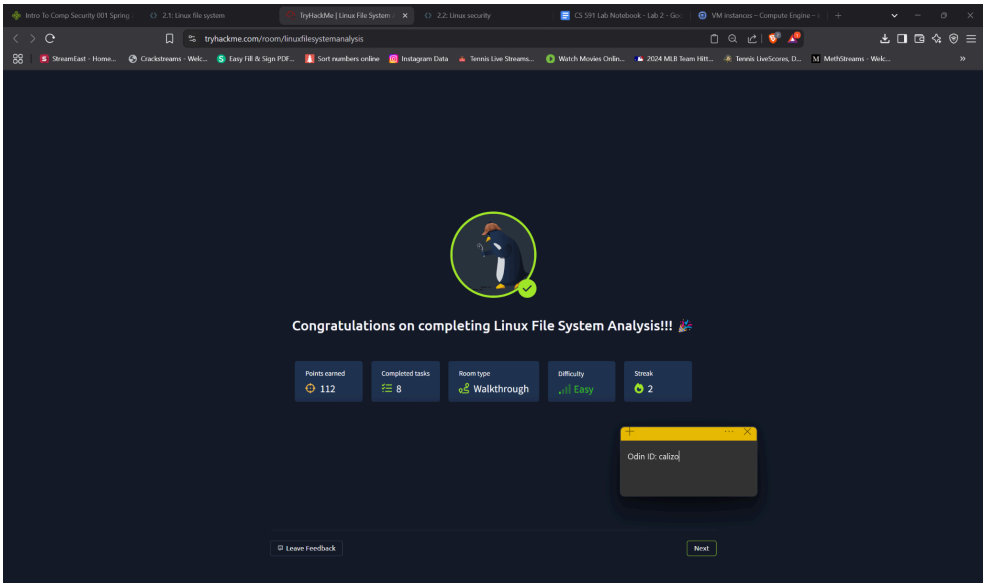
TryHackMe Linux File System Forensics

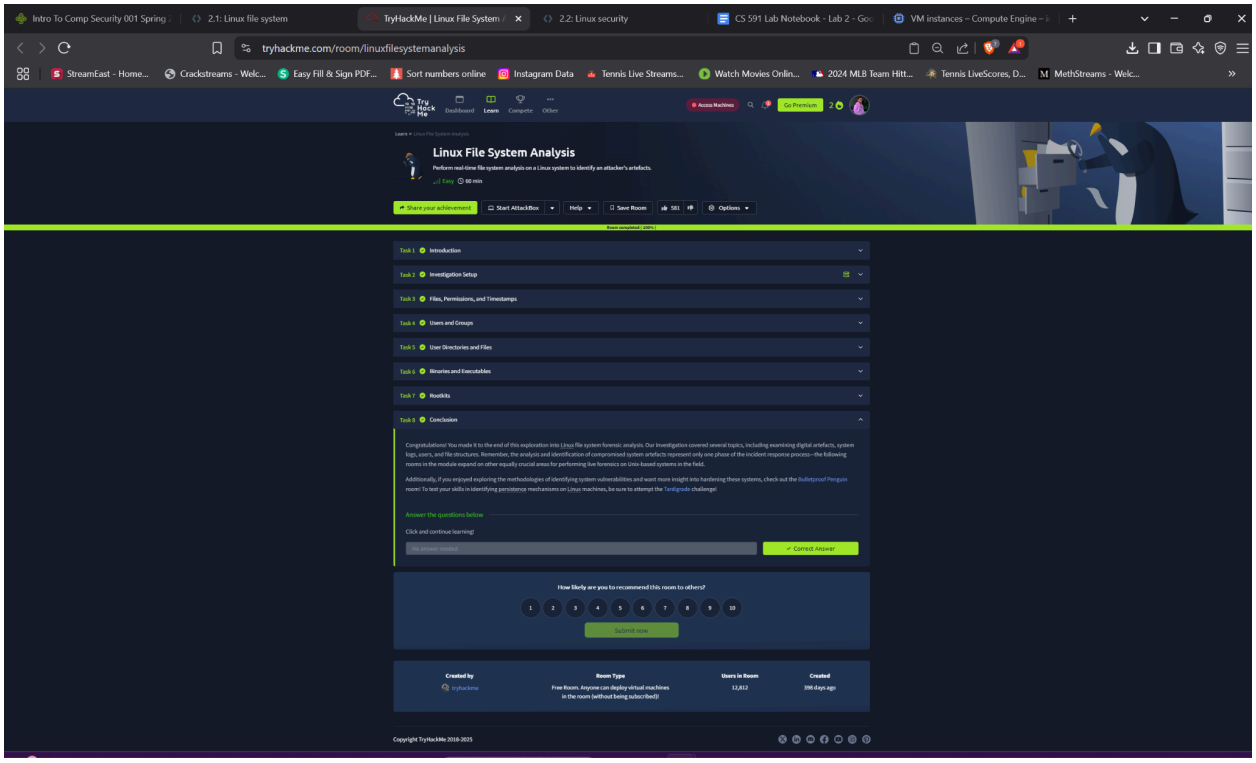
To round off Lab 2.1, I completed the Linux File System Analysis room on TryHackMe, which offered a comprehensive walkthrough of live forensic analysis techniques used during incident response on compromised Unix-based systems. The room covered a wide range of investigative methods, from locating rogue uploaded files and improper user permissions to identifying privilege escalation vectors and persistence mechanisms.

Throughout the room, I practiced applying various Linux commands like find, stat, sha256sum, md5sum, exiftool, debsums, chkrootkit, and rkhunter to analyze metadata, verify file integrity, uncover misconfigurations, and spot backdoor implants. I traced an attack chain from an initial file upload vulnerability to the creation of a malicious .phtml web shell, culminating in an SUID-enabled Bash shell for privilege escalation. I also identified a backdoor account and tracked unauthorized SSH key persistence through world-writable authorized_keys.

To wrap up, I ran rootkit detection tools (chkrootkit and rkhunter) on a mounted clean environment to ensure binary integrity. The lab reinforced the importance of using known-good binaries during live analysis and carefully preserving timestamp metadata. These exercises solidified key forensic principles and emphasized how small permission missteps—like SUID abuse or insecure SSH key management—can snowball into full system compromise.

The final TryHackMe completion page confirms full task completion and serves as documented proof of hands-on experience with live Linux incident response in a forensic context.

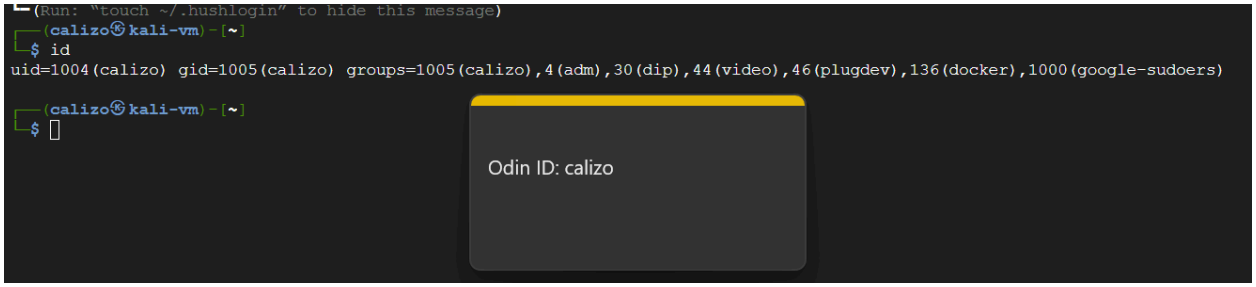




Lab 2.2 Linux Security

Account Creation, UID/GID Enumeration, and Privilege Check

To kick off Lab 2.2, I first verified my own account’s UID, GID, and associated group memberships using the id command. My main user calizo has **a UID of 1004 and a primary GID of 1005**. As expected for a preconfigured cloud-based Kali image, I also saw group memberships like adm, sudo, docker, and google-sudoers, confirming my elevated privileges for administrative operations.



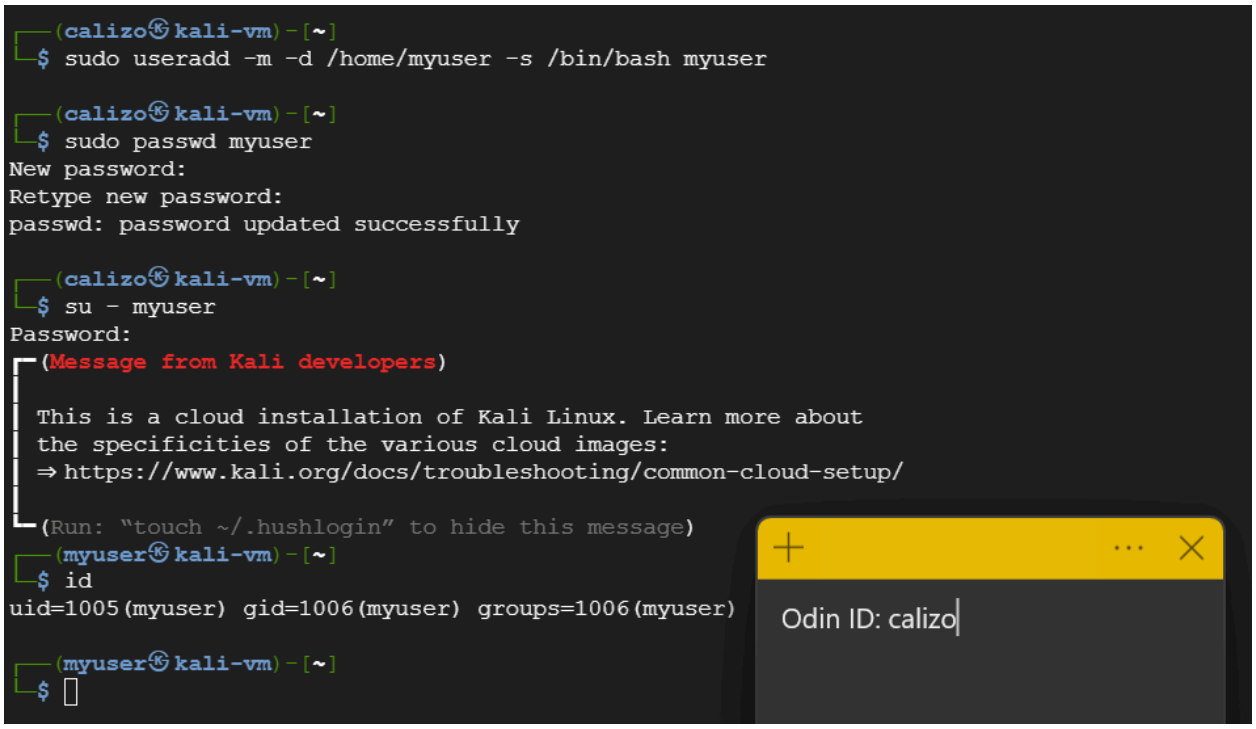
id
uid=1004(calizo) gid=1005(calizo) groups=1005(calizo),4(adm),30(dip),44(video),46(plugdev),136(docker),1000(google-sudoers)

Next, using sudo, I created a new user named myuser with a designated home directory (/home/myuser) and specified /bin/bash as the login shell:
sudo useradd -m -d /home/myuser -s /bin/bash myuser

After successfully setting a password with sudo passwd myuser, I switched into the new user session using su - myuser.

Once in the myuser environment, I ran id again to verify its new UID and GID. The output confirmed that myuser was created with **UID 1005 and GID 1006**, and only belonged to its own group (myuser). This reflects default Linux behavior where useradd assigns the next available UID and also sets up a matching group with the same name and GID.

Output:
uid=1005(myuser) gid=1006(myuser) groups=1006(myuser)



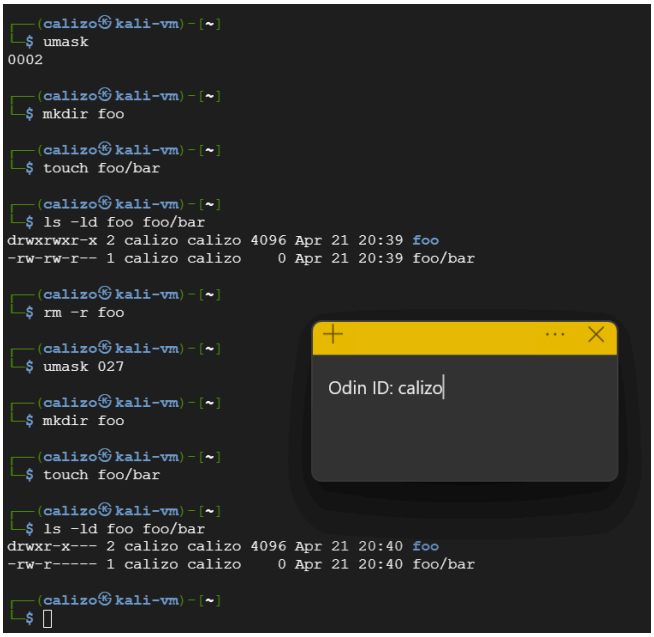
After verifying, I exited the session using Ctrl+D, returning to the main terminal as my original user.

This step reinforced the foundational Linux access control model, particularly the role UIDs/GIDs play in file ownership and privilege separation. It also reaffirmed best practices for creating users with limited group scope unless explicitly granted broader permissions.

Understanding umask and Default File Permissions

In this step, I examined the impact of umask on default file and directory permissions in Linux. The umask (user file creation mask) is a three-digit octal value that subtracts permissions from the system defaults—666 for files and 777 for directories—determining what access newly created files and directories will have. I first checked my current umask value with the umask command and found it to be 0002, which means the write bit is masked out for "others" by default. I tested this behavior by creating a new directory (foo) and a file within it (bar) using mkdir foo and touch foo/bar. After running ls -ld foo foo/bar, the directory showed drwxrwxr-x and the file displayed -rw-rw-r--, confirming the 775 and 664 permission outcomes based on the initial umask.

After cleaning up the test directory with rm -r foo, I modified the umask to a more restrictive setting with umask 027. The goal was to allow full access (read, write, and execute) to the owner, read and execute access to the group, and no access to others—equivalent to permission bits of 750 for directories and 640 for files. When I recreated the directory and file using the same commands and listed their permissions, the directory correctly reflected drwxr-x--- and the file -rw-r-----, aligning with the expected behavior for umask 027.



This exercise demonstrated how umask enforces least privilege by default and highlighted the importance of carefully configuring it in secure multi-user Linux environments.

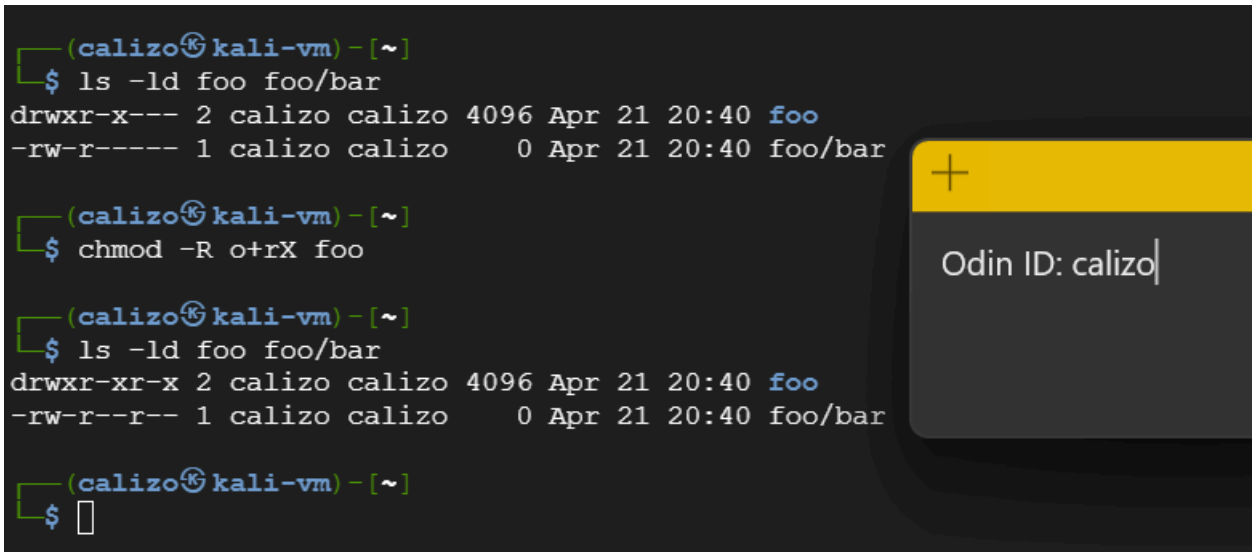
Modifying File and Directory Permissions with chmod

After configuring umask permissions in the previous step, I moved on to practice permission modification using chmod. I began by confirming the current access control settings on the foo directory and its file foo/bar using ls -ld foo foo/bar. As expected, foo had drwxr-x--- and foo/bar showed -rw-r-----, indicating limited accessibility to other users.

To adjust access rights and allow others to both read the file and traverse the directory, I issued the command:
chmod -R o+rx foo

This recursively granted read and execute permissions to the "others" group on all applicable entries under foo. The capital X variant is usually preferred when we only want to make directories executable, but since I wanted to allow execution on the directory and read access to the file, o+rx was the right fit here.

I validated the changes once more with ls -ld, which confirmed that foo now had drwxr-xr-x and foo/bar had -rw-r--r--, effectively granting others the ability to navigate into the directory and view the file contents without altering them.



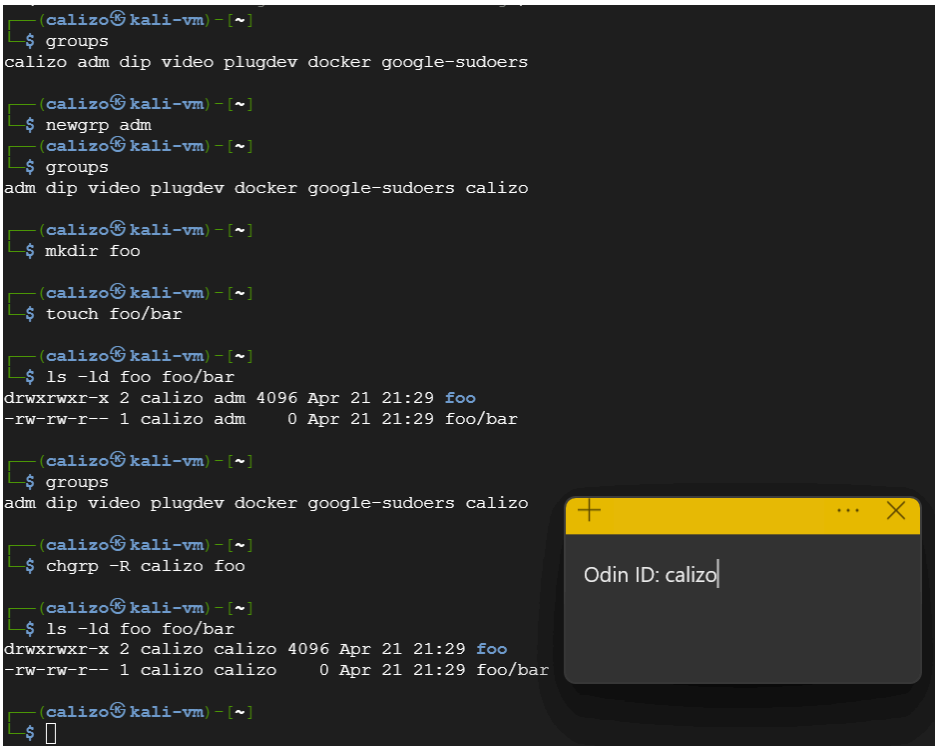
This hands-on step reinforced how Linux permission models operate at a granular level and how recursive permission changes should be applied carefully depending on the type of object (directory vs file) to avoid over-permissioning sensitive files.

Group Switching and chgrp Ownership Reassignment

In this step, I explored group management using groups, newgrp, and chgrp to observe how group assignments affect file creation and how to reassign group ownership on existing files. I first ran the groups command to list the groups my user account belongs to—calizo is my default login group, while others like adm, docker, and plugdev also appear.

I then used newgrp adm to switch my active group to adm and confirmed the switch by re-running groups, where adm became the first listed group. This change ensured that any files I created would inherit adm as the owning group. To confirm this, I recreated the foo directory and a test file foo/bar, then listed their metadata using ls -ld foo foo/bar. As expected, the user was calizo, but the group was now adm.

To revert the group ownership back to my default (calizo), I checked my active group list again with groups to verify the correct name, then ran chgrp -R calizo foo to recursively change group ownership of the entire foo directory tree. A final listing confirmed the change, with both foo and foo/bar now grouped under calizo once again.

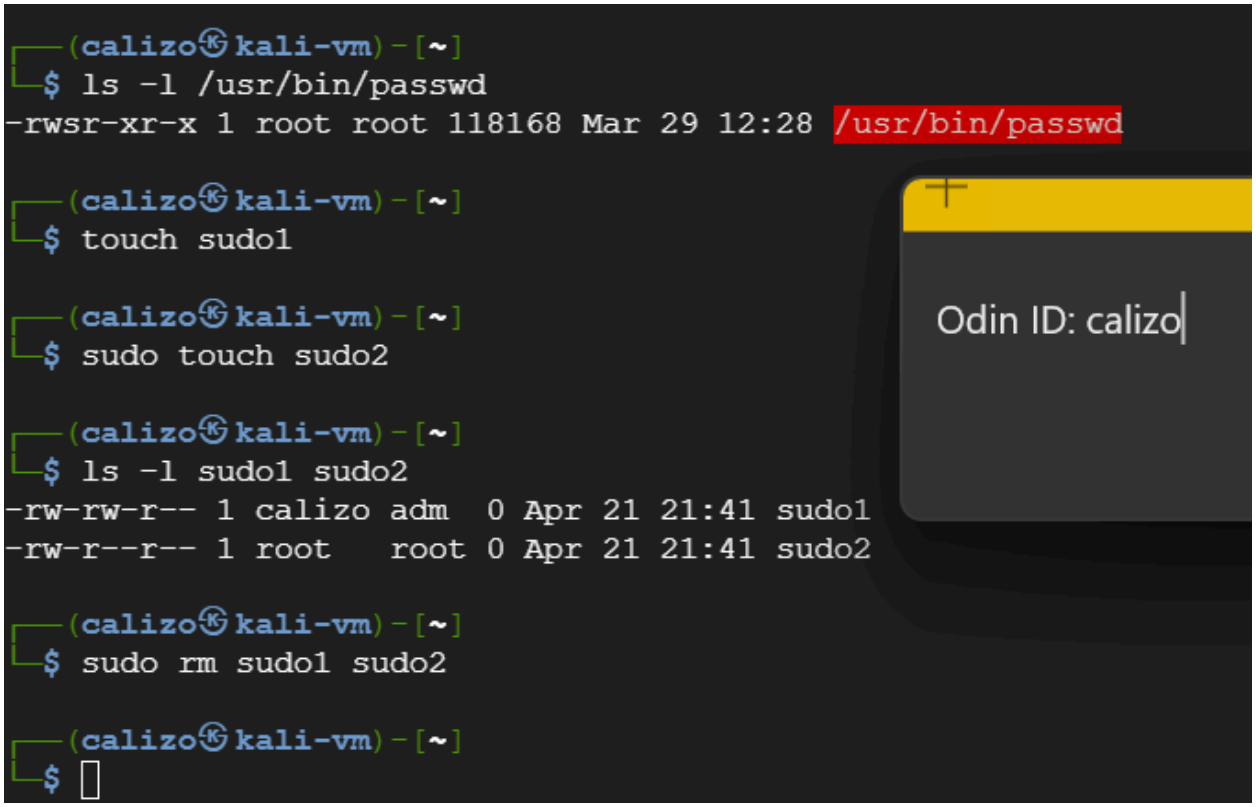


This exercise was a solid hands-on illustration of discretionary access control in Linux, showing how default group ownership is affected by login group context, and how it can be altered post-creation with chgrp.

Exploring setuid Permissions and Sudo Behavior

To deepen my understanding of Linux privilege escalation mechanisms, I examined how the setuid permission and sudo affect ownership and execution rights. I first listed the metadata for /usr/bin/passwd, a classic example of a setuid program. The permissions line confirmed that it indeed had the setuid bit set, shown by the rwsr-xr-x — meaning it executes with root privileges regardless of who runs it. This emphasizes why programs like passwd must be secure, as flaws in setuid binaries can be leveraged for full system compromise.

Next, I demonstrated how file ownership is affected by normal execution versus sudo-based execution. I created two files — sudo1 using a regular touch command, and sudo2 using sudo touch. Running ls -l on both showed that sudo1 was owned by my account (calizo), while sudo2 was owned by root, confirming that commands run with sudo inherit root's ownership and permissions. This aligns with how privilege escalation works temporarily under sudo without needing a persistent shell.



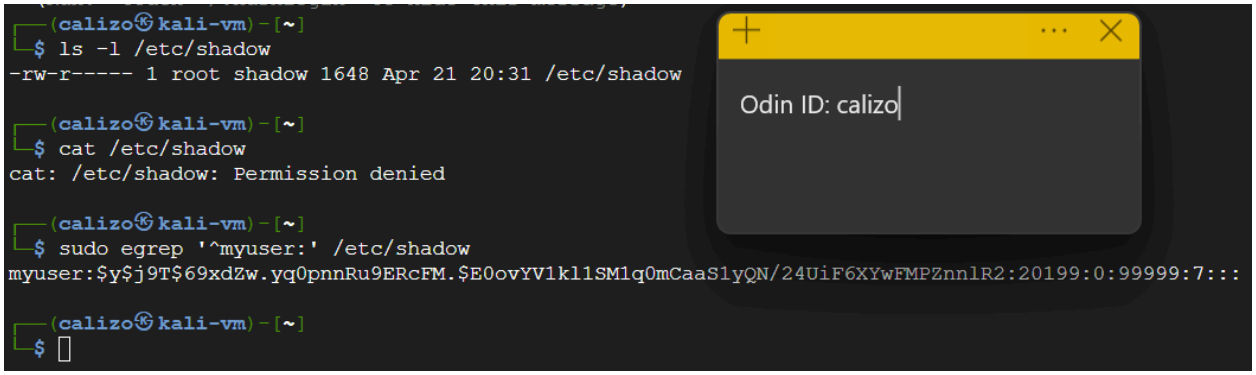
Finally, I used sudo rm to clean up both files, since sudo2 was owned by root and not removable with standard permissions. This exercise reinforced the practical implications of the setuid bit and how sudo grants elevated rights in a controlled way.

Investigating Shadow File Permissions & Using sudo/chown

In this step, I explored how Linux enforces privilege boundaries around sensitive authentication files, particularly /etc/shadow, and reinforced concepts around ownership modification via sudo and chown.

To begin, I inspected the permissions on /etc/shadow using ls -l /etc/shadow. The output showed the file is owned by the root user and group shadow, with permissions set to -rw-r-----. This explicitly grants read-write access only to root and read-only access to members of the shadow group. Because my user is not part of that group, the cat /etc/shadow command failed with a "Permission denied" error. This restriction is **intentional—it prevents regular users from accessing password hashes, which could otherwise be cracked offline**.

To demonstrate how an adversary with sudo privileges could escalate access, I ran the following command:
sudo egrep '^myuser:' /etc/shadow



This revealed the password hash entry for the myuser account, showcasing how sudo can be leveraged to read sensitive contents typically off-limits to normal users.

Next, I moved into the ownership manipulation section of the lab. I created a new directory and file with:
mkdir foo; touch foo/bar

Initially, these were owned by my current user. I then changed ownership of both recursively to myuser by executing:
sudo chown -R myuser foo

Upon running ls -ld foo foo/bar, I confirmed the new ownership: both the directory foo and the file bar were owned by myuser, while the group remained calizo. This

exercise highlights how root privileges allow control over file ownership, which could be abused by an attacker for persistence or obfuscation.

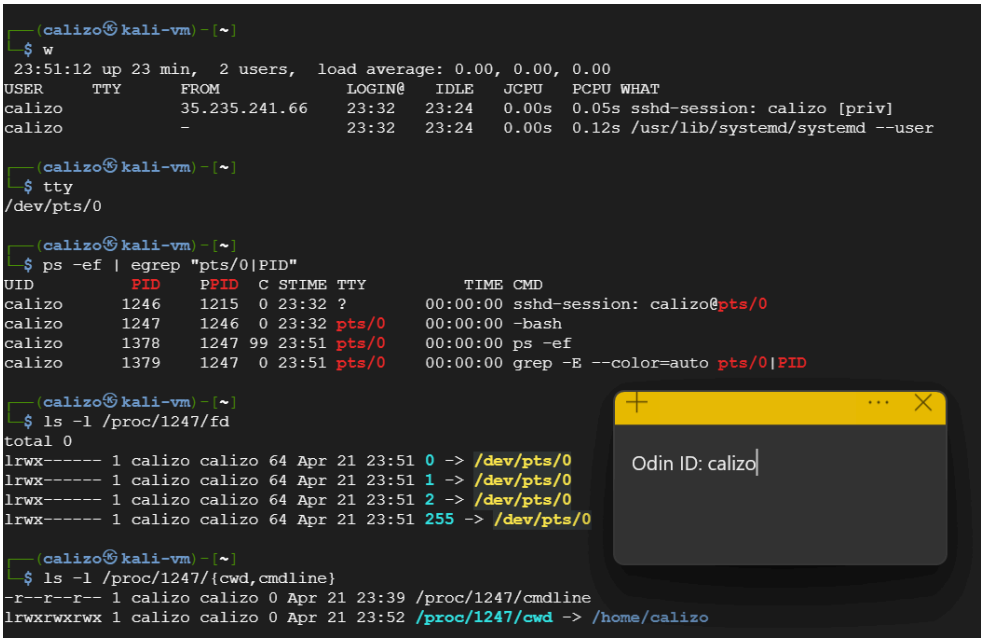


This step was a great reminder of how critical it is to tightly manage sudo access and monitor shadow file interactions in any hardened Linux system.

Processes and /proc Exploration

To begin exploring live process forensics on a potentially compromised Linux system, I first used the w command to inspect who was logged in. The output confirmed two sessions running under the calizo user. I followed this up with the tty command to identify my current terminal session, which returned /dev/pts/0. Knowing my TTY, I used the command ps -ef | egrep "pts/0|PID" to locate the active processes tied to this terminal. From the result, I identified my bash process with PID 1247 and its parent process (likely the SSH session) with PPID 1246.

To better understand how this shell process interacts with the system, I examined /proc/1247/fd, which lists all file descriptors opened by the bash process. Among the descriptors listed were 0, 1, and 2, which respectively correspond to **stdin, stdout, and stderr**—the standard input/output streams for any shell process. These map to the controlling terminal, /dev/pts/0, allowing command input and output flow for the shell.



I then queried the process's working directory and command-line arguments using: ls -l /proc/1247/{cwd,cmdline}

The output verified that the shell was running from /home/calizo, and the command-line file pointed to a bash instance.

Lastly, I viewed the process’s full status via less /proc/1247/status and **focused on the Umask line. The value listed, 0002**, indicates the default permission mask applied to newly created files. This value clears the write permission for "others," meaning any file this process creates will be readable by anyone but not writable by everyone—answering the question about global file readability in the process state.



Process Trees and Orphaned Execution in Linux

In this step, I explored Linux’s process hierarchy and parent-child relationships using tools like ps, pstree, and kill. To simulate a multi-process scenario, I created a custom shell script called sleep.bash containing four backgrounded sleep commands:

```
#!/bin/bash
sleep 1000 &
sleep 2000 &
sleep 3000 &
sleep 4000
```

After granting it execute permissions via chmod +x sleep.bash, I executed it in the background using ./sleep.bash &, which returned the PID 1418 for the parent script.

To investigate the script and its subprocesses:
ps -ef | egrep sleep.bash
showed the sleep.bash process itself. Then, using:
ps -ef | egrep sleep

I observed all four sleep processes running, each with their own PID. Their PPID was shown as 1418, which is the PID of the sleep.bash script. This confirmed that these were indeed subprocesses spawned from the script.

1. Why do the 4 sleep commands appear in this listing?
They appear because they are active background processes that were forked by the parent shell script. Each sleep command runs as a separate child process under the main script.

2. What does the parent ID of the sleep.bash process correspond to?
The PPID of sleep.bash was 1247, which is the PID of the current interactive Bash shell I was working in. This confirms that my shell session launched the script as a subprocess.

To better visualize the relationship, I ran:
pstree

This displayed a branching structure where bash had launched sleep.bash, which in turn spawned four sleep children processes. They were shown indented beneath the script process in the tree view — reaffirming the process ancestry.

I then terminated the sleep.bash script using:
kill 1418

After that, I reran pstree, which showed the absence of sleep.bash, but the four sleep processes remained alive.

3. What process is now the parent for the 4 sleep commands?
At the time of my ps snapshot, the PPID for each sleep process was still listed as 1418 (the now-defunct sleep.bash script). Normally, these orphaned processes would get reparented to the init process (PID 1), but this reparenting had not occurred yet, likely because the script process had not been fully reaped by the system at the time of inspection.

This highlighted a key behavior in live systems: even after sending a termination signal, reparenting can be slightly delayed depending on when the process is cleaned up.

4. What do their new PPIDs correspond to?
Again, the PPIDs still pointed to the original sleep.bash script (PID 1418) — meaning the kernel had not yet reassigned their parent to init when ps was run. If I had waited a bit longer or rerun ps, the PPIDs likely would have changed to 1.

Finally, I used killall sleep to terminate all the sleep processes in one go, successfully cleaning up the background processes and verifying control over multiple PIDs via process name matching.

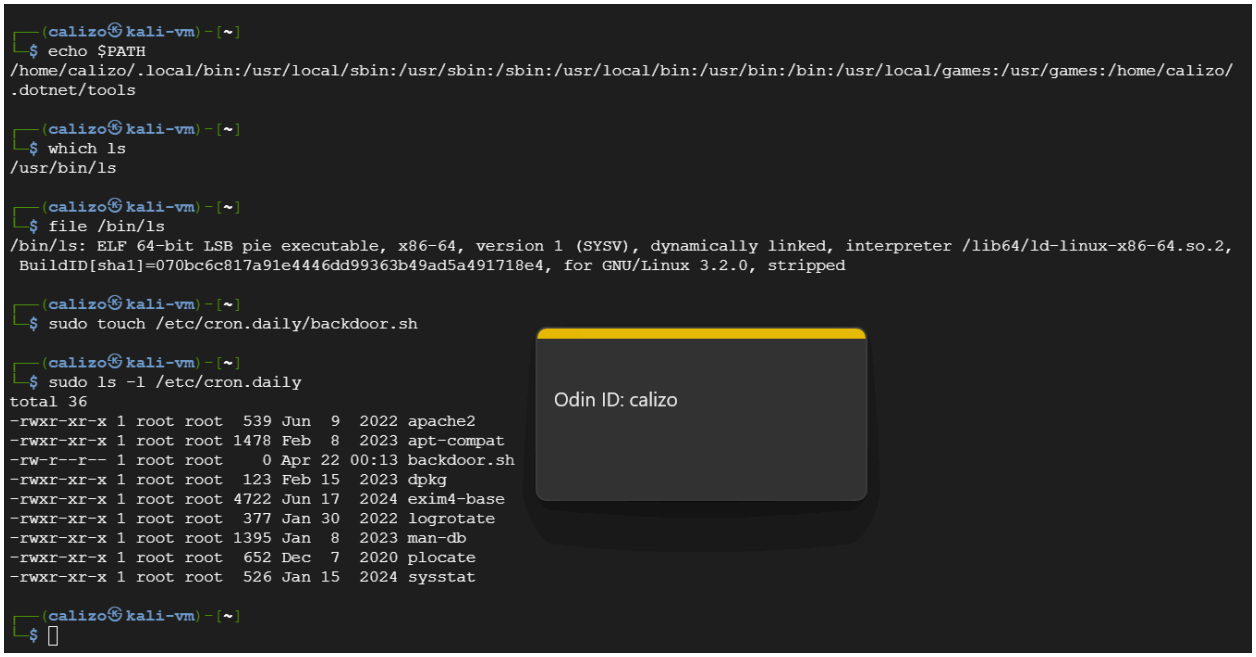
This step was a great way to reinforce how Linux process inheritance works, especially in the context of orphaned processes, backgrounding scripts, and command-level introspection.

PATH Hijacking and Cron Script Evasion

This step centered on exploring a classic misconfiguration vulnerability in Linux systems: PATH variable hijacking. First, I examined the current state of my environment by printing the contents of the $PATH variable. The output showed a typical structure where executable directories are searched in a specific order, starting with user-local and system-wide locations such as /home/calizo/.local/bin, /usr/local/sbin, and /usr/bin. I then ran which ls to confirm that the shell resolved the ls command to **/usr/bin/ls**, which is the expected default binary. To ensure the

binary wasn't tampered with, I used the file command and verified it as **an ELF 64-bit LSB executable—standard for core Linux utilities**.

Next, I began setting up a simulated cron-based backdoor. Using sudo, I created a file named backdoor.sh under /etc/cron.daily. Listing the directory (ls -l /etc/cron.daily) confirmed the file’s existence and its permissions (-rw-r--r--) showing ownership by root, as expected from a secure system-level scheduled task.



To simulate a PATH-based trojan scenario, I crafted a fake ls script under /usr/local/sbin—a location that appears in my $PATH before /usr/bin. This fake script echoed "OWNED by calizo" and invoked /bin/ls with the --hide "backdoor.sh" argument to stealthily omit the malicious script from listings. After saving and marking the script as executable with chmod +x, I confirmed via which ls that the system now recognized /usr/local/sbin/ls as the default. To avoid shell caching issues, I ran hash -r to refresh Bash’s command lookup hash table.



Upon re-listing the /etc/cron.daily directory, the output now excluded backdoor.sh and displayed the tampered message "OWNED by calizo", confirming that my trojanized ls was active and effectively hiding the file from unsuspecting users. This illustrates how improper PATH ordering or reliance on non-absolute paths can result in command hijacking, particularly if an attacker has write access to early-path directories.

sshd - SSH Configuration Analysis

To understand potential SSH-related misconfigurations that could be exploited in brute-force or privilege escalation scenarios, I began by reviewing the system's sshd configuration. I first accessed the man page for sshd to understand where the config file is located. According to the documentation, the file resides at /etc/ssh/sshd_config.

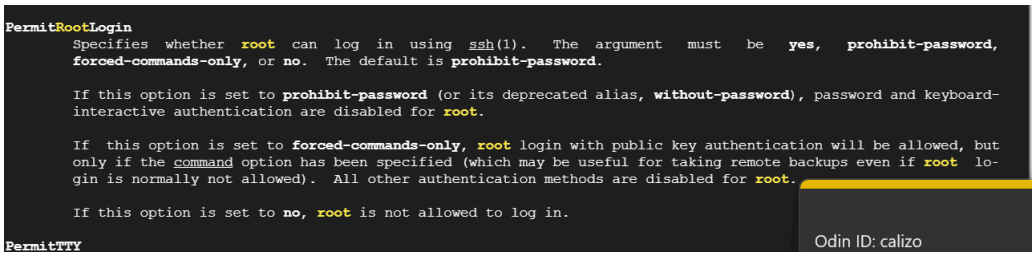
To locate key directives, I filtered for options relevant to secure login behavior:
sudo cat /etc/ssh/sshd_config | grep -Ei 'permitrootlogin|passwordauthentication'

The output confirmed the following:

- PermitRootLogin no
- PasswordAuthentication yes

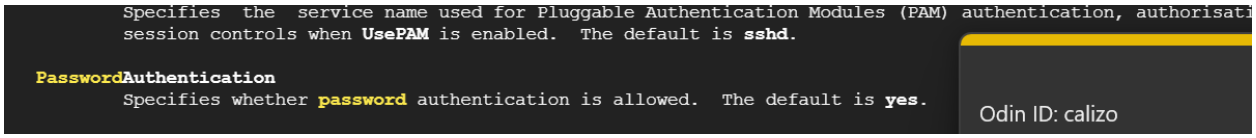
1. What is the name of the option that allows one to login as root?
   - **PermitRootLogin**

This directive controls whether the root user can log in directly via SSH. Setting this to no (as it is here) helps enforce the principle of least privilege by requiring users to escalate via sudo rather than logging in as root directly.

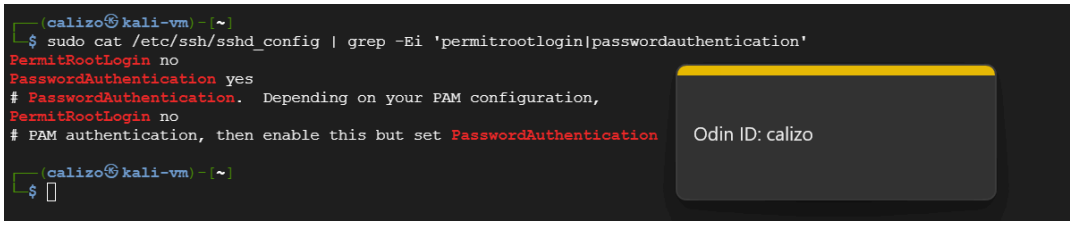


2. What is the name of the option that permits authentication via password?
   - **PasswordAuthentication**

This option specifies whether users are allowed to authenticate using passwords. In this case, it's set to yes, which is generally considered less secure compared to SSH key-based authentication—particularly in environments vulnerable to brute-force login attempts.



I cross-verified these values by also inspecting the related man page sections. The explanation for PermitRootLogin showed that if set to no, all forms of root login (including key-based) are disabled. The entry for PasswordAuthentication confirmed that this directive controls whether password-based logins are allowed, and the default is indeed yes.



Together, these values suggest a partially hardened configuration—root logins are disabled, but password-based logins are still open, leaving the system susceptible to brute-force attacks unless strong passwords or other protective controls (like fail2ban) are enforced.

Linux Process Analysis

This TryHackMe room walked through key techniques in process and application analysis on a Linux machine to uncover attacker persistence mechanisms. I approached this lab as both a refresher on Linux internals and a practical chance to apply digital forensics methodologies.

Task 1: Introduction
This task laid the groundwork for the lab's context—emphasizing the importance of identifying suspicious or persistent processes on compromised Linux systems. It also outlined the attack scenario and defensive objective: detecting artifacts left by a potential attacker.

Task 2: Investigation Setup
After launching the target Ubuntu 20.04 instance (IP: 10.10.108.103), I connected via SSH as investigator@tryhackme and started initial recon. I verified the environment's update status, listed user login history, and located the first flag by simply enumerating the home directory contents. File dumpzilla.py and another string on line 47 hinted that persistence would likely be uncovered through deeper inspection of process and startup mechanisms.

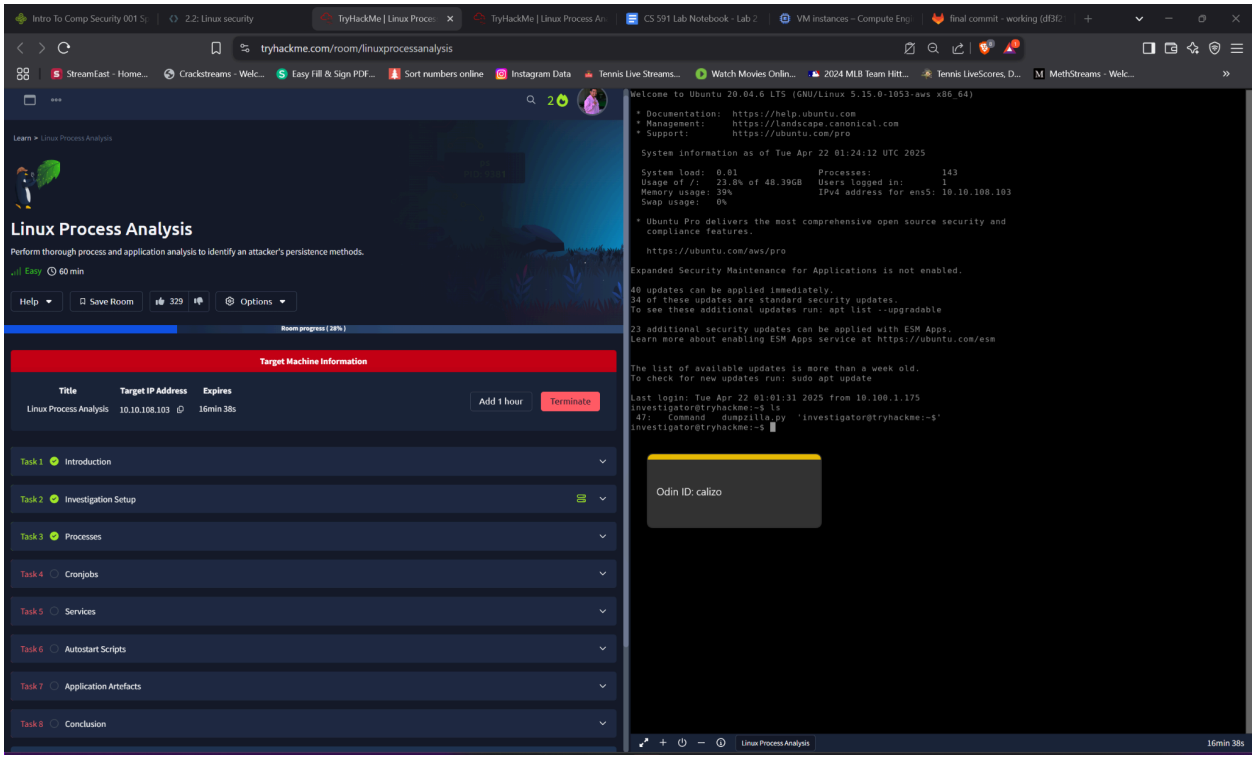
Task 3: Processes
I used standard Linux utilities such as ps aux, top, and pstree to list running processes. The objective here was to distinguish normal user-space processes from potentially malicious or obfuscated ones. I paid particular attention to:

- processes running as root
- those with unexpected parent-child relationships
- scripts with odd execution paths or timing patterns

This analysis helped establish a baseline of system behavior, although I didn't observe anything blatantly malicious within ps at this point.

Task 4: Cronjobs
I hit a roadblock here. The prompt asked for the full URL of a C2 server linked to a suspicious cronjob entry. I knew to check system-level crontabs like:
cat /etc/crontab
ls /etc/cron.*
crontab -l

I attempted to grep for wget, curl, or HTTP patterns that might suggest command-and-control callbacks but didn’t yield any meaningful matches. It’s likely I either missed a hidden script in /etc/cron.d/ or overlooked an encoded cron syntax.



I paused my analysis here due to time constraints, but I’ll be revisiting this task later with fresh eyes (possibly even scripting a wider cron audit next time).

Lessons So Far

- Process inspection alone rarely reveals the full picture—you have to trace those processes back to how and when they’re invoked.
- Crons and startup scripts are goldmines for long-term attacker footholds.
- The structured methodology—starting with the basics and peeling back layers—aligns well with the incident response mindset I’m building in CS 595.